



M363 Unit 11
UNDERGRADUATE COMPUTING

Software engineering with objects



Product quality:
metrics, verification,
validation, testing

Unit **11**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed in the United Kingdom by Thanet Press Ltd, Margate.

ISBN 978 0 7492 1617 7



1	Introduction	5
2	Software quality	6
2.1	What is quality?	6
2.2	Software quality factors	7
2.3	Summary of section	11
3	Verification, validation and testing	12
3.1	Verification and validation	12
3.2	Testing	15
3.3	Summary of section	25
4	Measuring software quality factors	26
4.1	Measuring quality and complexity	26
4.2	Metrics and quality measurement	26
4.3	Measuring system complexity	29
4.4	Summary of section	35
5	Testing techniques	36
5.1	Black box and white box testing	36
5.2	Ensuring testing is adequate	44
5.3	Summary of section	47
6	Formal technical reviews	48
6.1	Inspections	48
6.2	Making the most of inspections	51
6.3	Summary of section	52
7	Summary	53
	References	54
	Index	55

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are also due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

Introduction

In this unit, and in *Unit 13*, you will study quality and its relationship to the products and processes of the software development process. After setting the quality scene, this unit concentrates on software product quality. In *Unit 13*, we will return to quality issues in the context of the development process.

This unit explains the meaning of quality and shows how it can be measured, and discusses its importance within the software development process. We introduce verification and validation as the two main processes for achieving a quality software product. Finally, we show how testing and formal technical reviews underlie the verification and validation processes.

2

Software quality

This section begins with a short introduction to quality and its relationship to the products and processes of the software life cycle. Quality is related to the customer's requirements, and this allows us to provide working definitions of what factors affect product quality, together with ways of measuring them.

2.1 What is quality?

Gordon Moore of Intel proposed 'Moore's Law' that the number of transistors on an integrated circuit doubles every two years. The practical effect is that the price/performance of hardware follows the inverse curve; halving every two years.

Early in the history of the computing industry, the major cost component of computing systems was the development of the hardware upon which the software would run. Hardware costs have decreased quite rapidly from the late 1950s and, today, the major cost component is the development of the software. Mass marketing and the opening of large consumer bases for sophisticated computer-based products – such as videos, mobile telephones, washing machines, cars, computer games, etc. – have provided a vast and highly competitive market for software. This market has recently matured to the point where product *quality* – such as ease-of-use or reliability – provides a valuable commercial advantage.

But what, precisely, does 'quality' mean? When we talk about 'a *quality* car', we're distinguishing the car for its *high* quality, or excellence. On the other hand, when we ask about the '*qualities* that a person will bring to a job', we're talking about how well their *characteristics* will fit the role. Quality applies in both these ways to software too.

We would certainly require software to have the qualities or characteristics that make it fit for its purpose. We would also like the software to be of the highest quality possible. However, achieving high quality takes time and can be very costly. Just look at Rolls-Royce or Aston Martin – renowned for the high quality of their cars. However, neither Rolls-Royce nor Aston Martin works in a mass market, and to their customers the price paid for the high quality is only of minor importance (a phrase often used is 'if you have to ask how much it costs, you can't afford it'). Within a mass market (where most software ends up), quality only needs to be sufficiently high, or else its cost might exceed what the customer is willing to pay.

Software that is fit for its purpose and is of sufficiently high quality is said to be of *appropriate* quality, that is to say, it is in conformance to the requirements and expectations of the customer. We shall use the term *quality*, in the context of software, to mean 'of appropriate quality'. Some people use the expression 'good enough is good enough' to express the notion of appropriate quality.

As you will have noticed from your study of this course, a customer's requirements for software can take many forms. In general, they appear in a requirements statement that is part of the contract between developer and client. They can also arise and be documented during the iterations of the software development process. In either case, a customer's requirements are usually recorded in some way.

A customer's expectations, on the other hand, may or may not be recorded. Recorded expectations may stem from a knowledge of documented quality standards. Unfortunately, the customer may not be aware that they have expectations unless

these expectations fail to be met in the delivered product. These may stem from general characteristics that are expected of professionally developed software.

Despite the difficulty over being sure of a customer's requirements, the above definition of quality is a good working definition for practising software engineers, in that it allows them to decompose quality into measurable attributes of systems.

To establish whether quality is appropriate, we can check:

- ▶ how the product operates against what was both explicitly required and implicitly expected by the customer;
- ▶ that quality and development standards have been followed, whether they be in-house rules or ISO 9000 mandated practices;
- ▶ that best practice of software engineers, as professionals operating to professional standards, has been followed during development.

In this unit, we shall concentrate on what our definition of quality implies for a software *product*. *Unit 13* will concentrate on what our definition implies for the software development *process*.

2.2 Software quality factors

McCall has written very widely on the subject of software quality. In a joint paper, McCall and Cavano (1978) identified three general types of requirements – either stemming from the customer, or from other sources – that impinge on software product quality; these types still stand today. The types are product operation requirements, product revision requirements and product transition requirements. In addition, McCall described which attributes of a software product, or **software quality factors (SQFs)** as he termed them, are affected by these three types of requirements.

Product operation requirements concern how the product will be used. SQFs that are affected by product operation requirements are:

- ▶ *correctness*, which determines how well a system fulfils the customer's overall objectives, that is how well the software does what the customer wants;
- ▶ *reliability*, which determines the likelihood with which a system can be expected to perform its intended function, that is how well the software does what it is supposed to do;
- ▶ *efficiency*, which determines the level of computing resources (including time) required by a system to perform its function, that is how well it runs on the customer's hardware;
- ▶ **integrity**, which determines the extent to which access to data by unauthorised persons can be controlled, that is how well the data is secured;
- ▶ *usability*, which determines the effort required to learn about, to operate, to prepare input for, and to interpret the output of a system, that is how easy the system is to use.

Product revision requirements concern how the product will be changed. SQFs that are affected by product revision requirements are:

- ▶ *maintainability*, which determines the effort required to understand, locate and fix errors in a system, that is how easily bugs can be found and fixed;
- ▶ **flexibility**, which determines the effort required to modify an operational system, that is how easily the system can be changed while in service;

- ▶ **testability**, which determines the effort required to test a system to ensure that it performs its intended function, that is how easily the system can be tested to show that the customer's requirements have been met.

Product transition requirements are requirements relating to how the product will be modified for different operating environments. SQFs that are affected by product transition requirements are:

- ▶ **portability**, which determines the effort required to transfer the system from one hardware platform and/or software environment to another, that is how easily the system can be used on another machine should the customer change their platform or should other customers require it;
- ▶ **reusability**, which determines the extent to which a system (or system component) can be reused in other applications, that is how easy it is to reuse some of the software to make future developments more cost-effective;
- ▶ *interoperability*, which determines the effort required to couple one system to another, that is how easy it is to interface the system with another, should the customer require it.

For a particular development, some of the SQFs may be of more relevance than others. For example, a web server will probably require high efficiency and, depending on the nature of its contents, high integrity; a safety-critical aero-engine controller, on the other hand, will require correct and highly reliable performance, with efficiency a secondary concern, and no integrity demands.

McCall's classification is supported by the international standard ISO 9126 *Software engineering — Product quality*. This specification categorises software quality attributes into six characteristics: functionality, reliability, usability, efficiency, maintainability and portability.

SAQ 1

Suggest three expectations that a customer might have of a software product without, perhaps, being aware of them.

ANSWER.....

Three possibilities are:

- ▶ the product will not conflict with other software that they use;
- ▶ the product will boost productivity;
- ▶ the product will be simple to use.

SAQ 2

Explain how increasing integrity within a system could affect efficiency. Identify one other pair of SQFs which are not independent.

ANSWER.....

Increasing integrity within a system means making the system more secure. This might involve the use of passwords to access certain data and an authentication server to check a user's identity, or it might mean that network traffic needs to be encrypted and decrypted. Each of these factors adds overhead to processing, so efficiency is likely to be reduced.

Another pair of SQFs which aren't independent are usability and portability. Many of the features of the Apple Macintosh that contribute to its reputation for usability are built into its operating system. Applications that take advantage of these features are less portable to other systems, such as Windows or Linux.

Primary software quality factors

For everyday software products, Tom Gilb (1988) has identified correctness, integrity, maintainability and usability as being the primary SQFs. If we wish to be able to measure the quality of a typical software product, we thus need some way of assessing the levels of these four attributes. You can learn about usability and its evaluation in the course M364 *Fundamentals of Interaction Design*. Assessment of the other three primary SQFs can be done using various techniques, some of which are covered in the following discussion.

A popular measure for assessing correctness is *defects per thousand lines of code* (*defects per KLOC*), where a *defect* is defined as a verified lack of conformance to requirements. Defects are reported by a user of a software product after the product has been released for general use, and are counted over a standard period of time, for example one month.

There is further discussion of defects and lines of code later in this unit. Defects are commonly referred to as 'bugs'.

Integrity is measured by considering the proportion of 'attacks' on a product as opposed to bona fide uses. Some examples of attacks are:

- ▶ *denial of service*, in which a legitimate user is prevented from accessing a service to which they are entitled;
- ▶ *man in the middle* and *replay*, in which false communications between computers are generated;
- ▶ *accidental attacks*, in which failures of the product or its users are responsible for the attack, as, for instance, when a previous privileged user doesn't log out properly from a terminal;
- ▶ *input attacks*, where data is chosen to cause an application to function incorrectly.

For a particular type of attack, $\text{threat}_{\text{attack}}$ is defined as the *likelihood that an attack of this type will occur* within a given time and $\text{security}_{\text{attack}}$ is defined as the *likelihood that an attack of this type will be repelled*. For each type of attack, both $\text{threat}_{\text{attack}}$ and $\text{security}_{\text{attack}}$ can be given values between 0 and 1, which can be calculated from historical data, such as that contained in a log file. The *integrity against attacks of this type*, $\text{integrity}_{\text{attack}}$, is defined to be:

$$\text{integrity}_{\text{attack}} = 1 - \text{threat}_{\text{attack}} (1 - \text{security}_{\text{attack}})$$

The *integrity of the software product* is defined to be the sum, over all attack types, of $\text{integrity}_{\text{attack}}$:

$$\text{integrity} = \sum_{\text{attack}} \text{integrity}_{\text{attack}}$$

If there are n different types of attack, then integrity will be a value between 0 and n ; the higher the value, the better the integrity.

In this formula $\sum_{\text{attack}}$ means do the summation over all possible types of attack.

Example 1

A web server has been running for a month. From the log files for that month we see that, of 2000 accesses, 100 attacks were made. Of these, 50 were denial-of-service attacks, of which 10 were successful, 25 were password guessing (of which none were successful) and 25 were accidental

attacks (caused by errors on the part of the user), of which 25 were successful.

To calculate $\text{threat}_{\text{attack}}$ and $\text{security}_{\text{attack}}$ for each type of attack, we can tabulate the data as in Table 1.

Table 1 Calculating the integrity of a web server

Attack type	Threat	Security	Integrity
Denial of service	$50/2000 = 0.025$	$40/50 = 0.8$	$1 - 0.025 \times (1 - 0.8) = 0.995$
Password guessing	$25/2000 = 0.0125$	$25/25 = 1$	$1 - 0.0125 \times (1 - 1) = 1$
Accidental	$25/2000 = 0.0125$	$0/25 = 0$	$1 - 0.0125 \times (1 - 0) = 0.9875$

The integrity of the server is thus $0.995 + 1 + 0.9875 = 2.9825$.

Note that for software connected to the Internet, the probability of many attacks is 1. Internet systems are virtually guaranteed to be probed by password guessers, for common software defects like cross-site scripting (XSS), and for the possibility of social-engineering attacks like 'phishing'. Developers of systems that connect to the Internet should pay special attention to **security**.

Unfortunately, there is no way to measure *maintainability* directly, and so we must measure it indirectly. A simple measure is **mean time to change (MTTC)**, which is the average of the times it takes to analyse a bug report, design an appropriate modification, implement the change, test it and distribute the change to all users. In general, the lower the MTTC (for equivalent types of changes), the more maintainable the software product is.

SAQ 3

Log entries in a web server indicate that, over the previous month, 113 attacks were made on the web server, but none were successful. Is the web server necessarily secure?

ANSWER.....

The web server repelled all attacks, so *security* is 1. From the formula, *integrity* is 1 (perfect). We get this from working out the formula:

$$\text{integrity}_{\text{attack}} = 1 - \text{threat}_{\text{attack}} (1 - \text{security}_{\text{attack}})$$

This would seem to indicate that the web server is entirely secure. However, all we really know is that the web server was secure against these particular attacks. If the culprits were to use some other kind of attack, or change the parameters on the attack they used, they might well succeed. It is also possible that successful attacks have been made using some other method, but there were no log entries of sufficiently suspicious activity to attract our attention, perhaps because the attacker deleted them.



Exercise 1

What does ‘quality’ mean in contexts other than software?

Solution

You may have chosen to discuss a number of things here: a quality car service, a comparison of picture quality between DVD and video tape, the quality of services provided by local government, even the way in which light is depicted (the quality of the light) in Monet’s paintings of London. Whatever you have chosen, consider whether there are measurable attributes relating to quality, or whether quality refers to something less tangible.

2.3 Summary of section

In a mature mass market, quality to an appropriate level is an important attribute for a product to possess. In this section, you have studied how quality applies to software, how a customer’s requirements and expectations define appropriate quality, and how some aspects of quality can be measured.

3

Verification, validation and testing

Verification and validation are ways of assessing whether a software product (or any product, for that matter) does what it is supposed to do; which is fundamental to quality. In this section we shall study the processes of verification and validation in some detail, to see what is required for the processes to be performed properly. In addition we shall look at testing, an important tool used during verification and validation. Testing can be done throughout the software development process.

3.1

Verification and validation

Like quality, there are many complementary definitions of *verification* and *validation*, each of which emphasises some different aspect. Useful technical definitions of verification and validation are the following.

Verification is the process of checking that a given system description *S* is self-consistent, and that system descriptions that are derived from *S* are consistent with *S*. For example, a class diagram is one description of a system, and the code is another. If the code is not consistent with the class model, then verification should fail. Barry Boehm (1984) put it concisely when he said that verification answers the question: 'are we building the product right?'

Validation is the process of checking that each system description is consistent with the customer's requirements – that the system does what the customers want it to do. Referring to Boehm (1984) again, validation answers the question: 'are we building the right product?'

This definition includes *implicit* requirements and expectations, which have never been written down. For this reason, validation is generally harder than verification.

The term *system description* in these definitions refers to any means by which the system is described. This will certainly include the requirements statement prepared by the development team in association with the customer, the source code and the executable code that is delivered to the customer. It will also include the system documentation, the UML designs, and so on, associated with each activity in the development of the software product.

How to carry out verification and validation

From our definition of verification, the task we set ourselves in the verification of a system is twofold:

- ▶ we need to make sure that all system descriptions are self-consistent;
- ▶ we need to make sure that all system descriptions are consistent and complete with respect to those from which they were derived.

Likewise, from our definition, for validation:

- ▶ we need to make sure that all system descriptions are consistent with the customer's requirements.

Consistency and self-consistency

That two system descriptions are consistent simply means, where they describe the same part of the system, they do not contradict each other. A simple example of inconsistency in this sense is when the documentation says of a software product that 'the print key can be used to print the screen', but this functionality is not present in the product itself.

Consistency with a customer requirement means that the requirement is met. For example, imagine we were validating a product against the customer requirement that:

The user should be able to search the names and addresses of 10 000 people in less than a second.

If the product took 10 seconds to do the search, then clearly we have an inconsistency. Consistency with some requirements can be difficult to check. For instance, take the requirement:

The system must take 35 per cent of the world market in point of sale systems.

The task of checking consistency is difficult, if not impossible!

A system description may be inconsistent with itself. This is the situation in which one part of the system description says one thing, only to be contradicted by another part. Such self-inconsistency is most likely to happen in the system documentation, in which contradictory claims of the system are often made. However, even an implementation need not be self-consistent. For instance, an attribute, `address1`, declared as:

```
Address address1;  
IPAddress address1;
```

This is self-inconsistent. In this case, the job of checking self-consistency is a task that can be performed by the compiler; such code would fail to compile. There are other inconsistencies that a compiler usually would not catch, such as one developer assuming that a unique ID generator is zero-based while another developer assumed it was one-based.

You may have seen a lack of consistency in the packaging for software. On the sealed envelope holding the CD-ROM one might find: 'Do not break this seal without reading the licence agreement', which is inside the sealed envelope!

Completeness

In contrast to consistency, the concept of **completeness** is very simple to grasp. It means that everything that should have been 'said' in a system description has been 'said'. Unfortunately, *showing* completeness of a system description is a very much more difficult task than showing consistency, for it means that *every aspect of the customer's requirements* must be met by the system description. This includes, in theory, requirements such as:

- ▶ the system must take 35 per cent of the world market in point of sale systems;
- ▶ the system must be secure;
- ▶ the system must be maintainable for 10 years.

Early in the development process, it is worthwhile reviewing the requirements statement for requirements that are impossible to verify or validate, with the aim of either improving the requirements or removing them from the requirements statement (after appropriate **review**). If one cannot establish *fit criteria*, measurements by which one knows that the requirement has been satisfied, then the requirement is a candidate for removal.

Fit criteria and functional requirements were introduced in *Unit 2*.

Because completeness is such a difficult property to ensure, practically all that one can hope to achieve is **analytical completeness**, where only requirements with fit criteria are considered. In theory, functional requirements, which are requirements that describe what the system is to do, have fit criteria; the system does it or it does not. Unfortunately, in practice, functional requirements do not cover *every* situation that might come up, so there are usually missing criteria. Non-functional requirements, which are requirements that describe qualities of the system, often do not have binary fit criteria (yes/no answers). Instead, the developers must discuss the issues with the customers to try to quantify what the customer means, and then use the results as fit criteria. Notations such as UML can be very helpful for showing completeness of functional requirements, by helping customers understand what the developers think that the requirements are.

The development process

In the next subsection we shall see that testing is one technique used for validation and verification. Traditionally, testing has mainly occurred very late in the development process. This approach has proved problematic, since we may find during testing that we have actually constructed the product wrongly (verification is not possible) and/or built a different product to that expected by the user (validation is not possible).

An alternative model of software development is to attempt to establish that we are constructing the correct product early in the development process. In *Unit 3*, we saw how the use of prototyping facilitates the identification of user requirements. This is an example of validation.

The use of prototypes supports a user-centred approach to the early phases of the development process. As we move through the development process, we will tend to gain a richer picture of the software we are constructing. As our software model becomes more complete, we should do further work on validation to pick up on requirements that may not have been clear earlier on. This should involve further user involvement, as we can never be sure our written user requirements are complete or that the users are able to provide precise descriptions.

Given a reasonable set of user requirements, verification techniques should be used during each activity in the development process to ensure that we actually implement them correctly. We need to ensure that the deliverable from each activity is consistent and complete with respect to the input to that activity. For example, if we construct a class diagram on the basis of a set of use cases, the classes must be exactly those needed to support the use cases being implemented. Notice that this does not preclude an incremental approach to development.

SAQ 4

Why is it important for the customer's requirements statement to be self-consistent?

ANSWER.....

If the customer's requirement statement lacks self-consistency, then either the resulting system will be inconsistent or it will not satisfy the customer's requirements. The system builders can decide (implicitly or explicitly) how to resolve the inconsistencies, or, if the inconsistent requirements affect different parts of the system, the developers could inadvertently build the inconsistencies into the product.

SAQ 5

Give an example of a product you know where the user manual or documentation is not complete.

ANSWER.....

Most user manuals are not complete, in the sense that they will not describe all the features of a product. For example, a word processor might contain many features that are not fully described anywhere. For example, two features might interact in unexpected ways.

3.2 Testing

Testing is the process of exercising software to check that it does what it is supposed to (and doesn't do what it is not supposed to). All software contains flaws and during testing we aim to detect as high a proportion of them as possible, so that they can be rectified before the software is released. In other words, it is any form of validation or verification that operates on program code.

Testing is just one tool in the software engineer's toolkit for validation and verification. It is, however, one of the most important tools in that toolkit. In this subsection we shall consider precisely why testing is useful, some of the basic techniques that are used in testing, and how they fit together to give a testing strategy.

The basic test procedure is to select some function of the software, some test data that the function uses, predict the results that should be expected from running or using the function, and then check whether the software conforms to our expectations. If not, then either we have misunderstood what the function was supposed to do, or more likely we have found a bug that needs to be fixed. It is usually completely impractical to test software for every possible input condition. Therefore, a range of techniques is used to attempt to find a relatively small set of test data that is in some way representative. Although, as you will see in Section 5, a set of sophisticated techniques are available to help us choose representative data, we cannot be entirely certain that testing will uncover all the bugs.

Thus the role of testing is to help find as many bugs as possible, given the resources available for testing. Of course, the reason that finding bugs is important is that by finding *and fixing* bugs, software quality can be improved.

Other than this though, testing can improve the customer's (and the developer's) confidence in a software product. Statistical techniques can be used to model the patterns of bugs found by testing different products, in order to estimate how many bugs there are likely to be in a particular product before testing starts. Then, armed with the knowledge of how many bugs have been found (and fixed) through testing so far, we can estimate how many bugs remain to be found. Such statistical estimation can be used to present independent arguments to the customer that the delivered software meets specified quality requirements.

Moreover, testing can improve the testing process itself. A cost-benefit analysis of 'bugs found' versus 'time taken' can be performed to determine when testing should stop, under the law of diminishing returns. Furthermore, a detailed examination of where bugs arise in code has shown that programmer-induced errors occur most often where code complexity is high. Used in combination with complexity measures (which you will study in the next subsection), testing can be optimised further by concentrating testing effort where the code is most complex.

Although this data is old, similar data could be collected for a statistical analysis of today's systems.

Example 2

Table 2 lists data collected by John Musa at Bell Laboratories in 1979. The data comes from the testing of a small component of a real-time command and control system for which Musa was project manager, and gives the execution time, column by column, between testing failures to the nearest second. The first entry, 3, in the table tells us that the system failed within about 3 seconds of starting. The system was then debugged (the error was removed) and the system restarted. The next entry, 30, tells us that the system failed again about 30 seconds after this restart.

Table 2 Execution times, in seconds, between successive failures for a computer-controlled system

3	88	36	193	0	529	724	1247	1071
30	670	4	6	232	281	2323	943	371
113	120	0	79	330	160	2930	700	790
81	26	8	806	365	828	1461	875	6150
115	114	227	1351	1222	1011	843	245	3321
9	325	65	148	543	455	12	122	1045
2	55	176	21	10	296	261	990	648
91	242	58	233	16	1755	1800	948	5485
112	68	457	134	529	1064	865	1082	1160
15	522	300	357	379	1783	1435	22	1864
138	180	97	193	44	860	30	75	4116
50	10	263	236	129	983	143	482	
77	1146	452	31	810	707	108	5509	
24	600	255	369	290	33	0	100	
108	15	197	748	300	868	3110	10	

Bev Littlewood (1992), a Professor of software engineering at City University, has pointed out that there are some striking features of the data in Table 2 that are typical of software failure data. There is obvious growth taking place in the reliability, since the time between failures generally increases as we progress through the data, as Figure 1 illustrates. However, though the reliability increases over time, the time between failures is still very variable right to the end: in the third from last column of Table 2, after about 42 000 seconds of testing, there is still a 0, that is a failure within half a second of restarting the system.

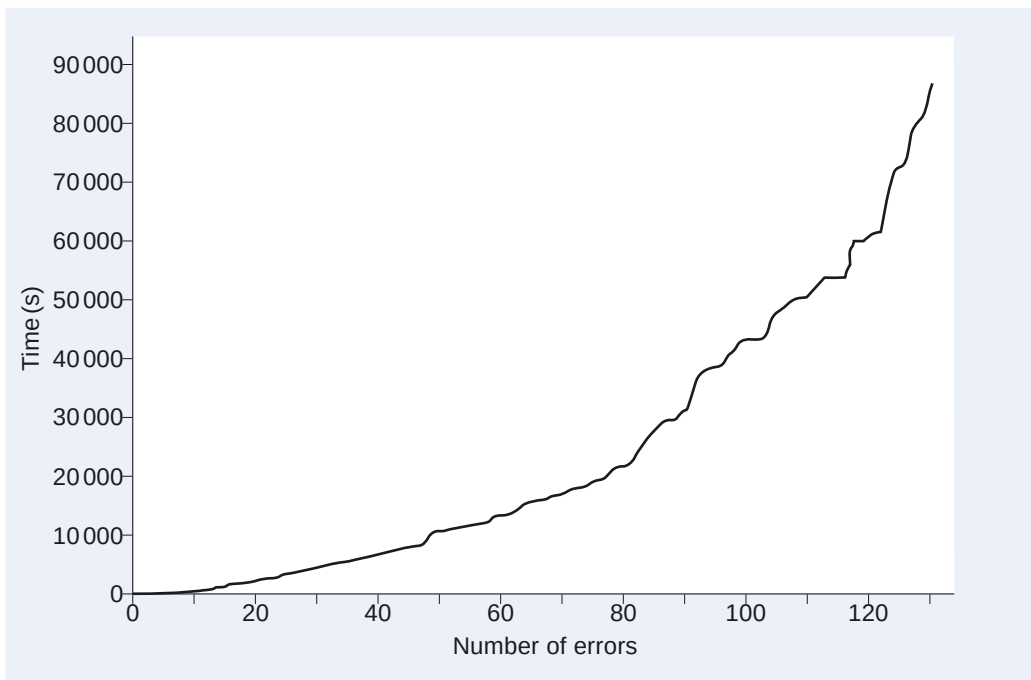


Figure 1 The growth in reliability for the data of Table 2, obtained by plotting when failures happened over time

So, even without complex statistical justification, you can see how testing allows us to predict reliability. The data in Table 2 shows that there were 131 failures during the testing process. Statistical estimation shows that there are probably around 10 or so bugs remaining in the system.

SAQ 6

Continue the trend of Figure 1 by extending the curve that appears there. Can you make any predictions about the number of errors that were originally present in the system?

ANSWER.....

The curve you have drawn may become near vertical at around 140–45 errors. If this is the case, then this indicates that the time between failures becomes very large in that range, meaning there are few errors left to be found. Consequently, the number of errors that were originally present was somewhere around 140–145.

The testing process

There are four distinct types of testing that occur during a software development project:

- ▶ **usability testing**, which is discussed in the course M364 *Fundamentals of Interaction Design*;
- ▶ **developmental testing**, which checks that developmental activities have been carried out correctly, and ideally it is performed by development staff other than the authors of the code and checked by quality assurance (QA) staff;
- ▶ **requirements-based testing**, which checks that a system meets the customer's requirements, and it is carried out by QA staff in conjunction with the customer;

- ▶ **regression testing**, which occurs during developmental testing and system maintenance, and checks that fixing one bug has not introduced others.

Testing is carried out throughout that part of the software development process dealing with executable program code.

Developmental testing consists largely of three stages:

- ▶ **unit testing**, in which units of functionality (for example, the classes in an object-oriented system) are tested in isolation;
- ▶ **integration testing**, in which previously unit-tested units are tested together;
- ▶ **system testing**, in which the completed system is tested against the customer's requirements.

Requirements-based testing consists largely of:

- ▶ **acceptance testing**, which is performed by the customer, and after which (all being well) the system is accepted.

SAQ 7

What do you think is the relationship between system and acceptance testing?

ANSWER.....

In general, the same tests will be carried out during acceptance testing and system testing. System testing is an in-house activity and a customer need never know how system testing went; any bugs can be dealt with before the customer sees them. Acceptance testing, on the other hand, is conducted with much more at stake; the customer can *accept or reject* a system based on its performance at acceptance testing.

Unit testing

Some authors prefer unit testing to be at the package level. We shall concentrate on the class as the unit. Techniques for the unit testing of packages can be easily extrapolated from those for classes.

In an object-oriented system the structural unit is the class; unit testing is therefore usually performed on classes. When we say that a class is tested, we include testing the methods in the class. The methods might be tested individually, but as we must create an instance of a class to test a method, we are also testing the class itself.

In unit testing, a class is tested in isolation to see whether or not it meets its specification. After test data has been found (in ways that you will study in Section 5) and criteria have been developed for measuring the success of the test (usually just what the expected answer is, derived from the specification), the unit is tested using the test data, and test results are collected. By comparison with the criteria, the results can be analysed to reveal the presence of bugs, which can subsequently be isolated and fixed. When the unit is modified to fix a bug, it must be rerun against the same test data to ensure that any modifications have not inadvertently affected other parts of the code (this is regression testing).

Where a class has been derived from another class, it will still need to be fully tested. We must not assume that inherited methods that have not been overridden will function as in the base class. This is because they may interact with other methods, whose behaviour has been modified.

For 'formal' unit testing, a programmer should not test his or her own program.

In practice, however, unit testing is often only a relatively informal process performed by the programmer during development.

For a particular class, unit testing should consist of five parts:

- ▶ **interface testing**, where test data is chosen to check that the flow of information into and out of an object of the class is correct;
- ▶ **boundary-condition testing**, where test data are chosen to check that methods perform correctly at the extremities of input ranges;
- ▶ *local-data testing*, where test data are chosen to check that methods manipulate attributes correctly, so that an object's state remains consistent;
- ▶ **control-structure testing**, where test data are chosen to check that algorithms are coded correctly and to execute each statement in the code at least once;
- ▶ *error-handler testing*, where test data are chosen to check that the object performs correctly in exceptional situations.

A challenge with testing a class in isolation is that objects of the class need to be instantiated to be executed, and so require a program to invoke them and to feed them with the appropriate test data. To do this, extra software, known as a **test harness**, is used. Sometimes this software is written specially; often, however, a general-purpose tool is used. The test harness provides values of the input parameters to invoke an object's methods, as well as the object's internal state before the test, and displays the values returned, as well as the object's state, after the invoked method has returned.

You will see an example of a test harness in Section 5 of this unit.

Another challenge with unit testing is being able to test a class which depends upon other classes for the functionality it provides. In this situation, **test stubs** – very simple pieces of code which stand in for the called method – may be used. A test stub will implement the same signature as the called method, but provide none of its functionality, printing only verification messages for the data sent to it and returning to the caller with the 'expected' value.

In object-oriented systems, however, test stubs are not always necessary; **bottom-up testing**, in which a class is tested according to its use and the *has-a* relationships in its class diagram, can be used. The basis of bottom-up testing is that, in general, the objects forming the *leaves* of the class diagram (that is, those classes providing services to other classes, but not using the services of other classes) can be tested first, without needing test stubs. Unit testing then proceeds according to the class diagram using the already-tested classes.

Leaves may not exist if there are cycles in the class diagram. In this case you must use stubs.

Bottom-up testing can do away with the need for test stubs in most cases. However, there are two other factors to consider. The first is when a class is linked with the outside world, perhaps monitoring some real-world device, when providing test stubs for the device is likely to be more cost-effective than requiring access to it. The second is that, as mentioned before, errors are more likely to exist in the more complex classes, and so it makes sense to concentrate the testing effort on these. In this case, test stubs may be needed, although there is a difficult balance to be struck.

Within a class, a similar balance needs to be struck between testing the methods that rely on no others for the services they provide, and concentrating testing effort on the most complex methods.

SAQ 8

A *getter* is a method that returns the value of one of an object's attributes. A *setter* sets the value of one of an object's attributes. What use could getters and setters be to unit testing?

ANSWER.....

Getters and setters allow the state of an object to be interrogated. Once the getters and setters have been tested (a process that may be as simple as verifying that the

getter returns what was just set, or much more complex if the getters and setters do calculations), they can be used in more complex tests, setting initial state and verifying final state. Being able to set the state before a testing a unit, and to interrogate it afterwards, are indispensable aspects of testing.

Integration testing

Integration testing builds on unit testing, by testing units in combination; essentially, partial builds of a software system are tested. Integration testing involves checking that unit-tested classes interface correctly together. In the case of software using a procedural language, there can be a need for a considerable amount of integration testing. However, in the case of software developed using an object-oriented language, the structuring of the classes and packages means that integration testing is less of a burden.

There is, however, one major problem with integration testing that is specific to object-oriented software, which is due to the complexities of dynamic binding caused by inheritance and polymorphism. The problem, illustrated in the example below, is due to the possibility of subtle coupling relationships between classes being combined with variations in behaviour due to polymorphism.

Example 3

Consider a base class, `PlaneFigure`, from which the classes `Triangle`, `Circle` and `Line` are all derived. We might define an abstract method `draw` in `PlaneFigure` to indicate that derived classes should be able to render themselves to the screen. `Triangle`, `Circle` and `Line` will implement `draw` as they wish.

Furthermore, suppose classes `A`, `B` and `C` have a `design(PlaneFigure aFigure)` method which invokes `aFigure.draw`. The dynamic binding of the `draw` method means that, at run time, any of three different `draw` methods could be invoked (one for each of `Triangle`, `Circle` and `Line`) for each of the classes `A`, `B` and `C`, so that nine possible run-time combinations exist. This is illustrated in Figure 2.

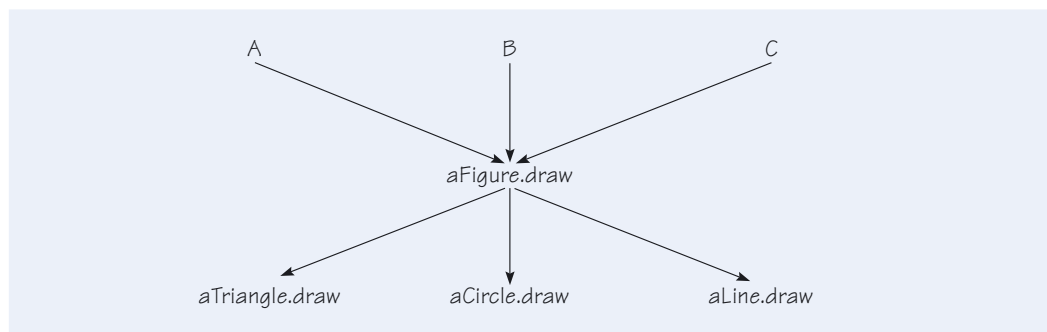


Figure 2 The `aFigure.draw` message from classes `A`, `B` and `C` can be resolved at run-time in any of nine different ways

Should we test all of the combinations? Conceivably, errors in the `Triangle` `draw` method could be uncovered only when the calling object is an object of class `C`, but not when called by `A` or `B`. This might happen, for example, if because of the requirements of the system, classes `A` and `B` need draw only right-angled triangles and therefore this is what is tested, but `C` draws triangles with arbitrary angles. In this case, it is only by testing the combination of `C` with `Triangle` that we discover the error.

In Figure 2, Figure 3, Figure 4 and Figure 5, an arrow represents invocation; the arrow between class `A` and `aFigure.draw` means that `aFigure.draw` is found in the code of method of `A`. However, because of inheritance, when `A` invokes `aFigure.draw` on an object `aFigure` that is actually a `Triangle` (perhaps by assignment e.g. `aFigure = aTriangle`), then the method which is executed is `aTriangle.draw`.

When errors are *dependent* in this way, the best testing strategy is to test all combinations; in this simple case, we could test all nine combinations and so be sure of testing the erroneous combination. This is, theoretically, the safest testing scheme as it is the only strategy for ensuring that all combinations of invoking and target classes are tested together, therefore being sure of testing dependent errors. This is illustrated in Figure 3.

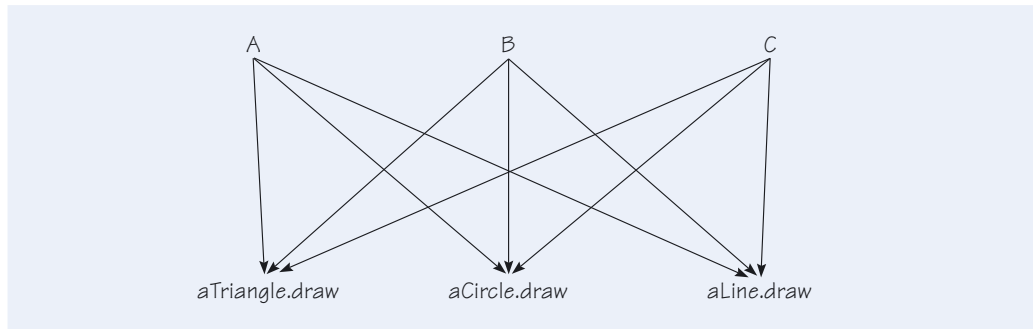


Figure 3 The theoretically best polymorphic testing strategy, in which all combinations are tested

In the general case, it is sensible to try to reduce the testing load in the presence of inheritance and polymorphism, while still retaining validity of the testing strategy. This can be achieved by testing only a subset of the combinations.

There are two main strategies. The first assumes that any errors in the invoking object and the target object are independent between invoking and target classes, and means that, for the above example, we need choose only three representative subcases, such as those illustrated in Figure 4.

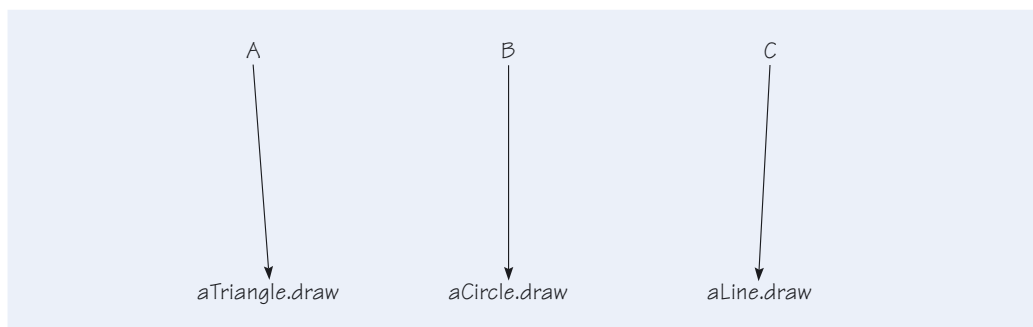


Figure 4 A polymorphic testing strategy in which errors are assumed to be independent between invoking and target classes

The assumption that errors are independent is, however, not always safe. A second strategy, which balances the need to test independence while not requiring full testing, is to choose *one* of A, B and C (say A) and test it with *all* of Triangle, Circle and Line, and choose *one* of Triangle, Circle and Line (say Circle) and test it with *all* of A, B and C. This is illustrated in Figure 5. Note that the combination of A with Circle is in both groups of tests but of course only needs to be carried out once.

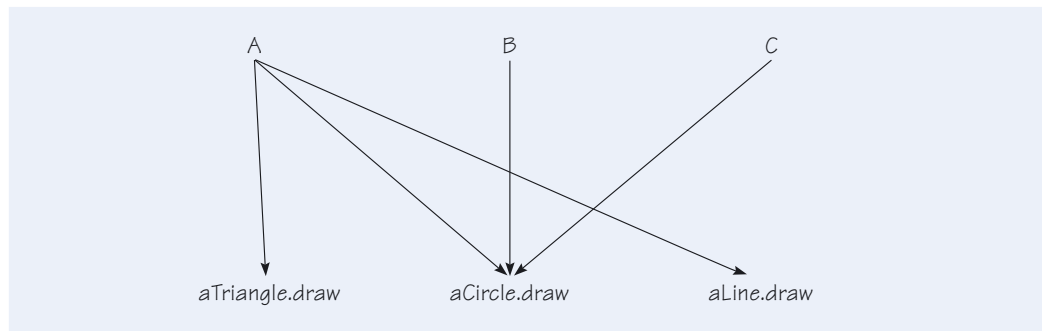


Figure 5 A polymorphic testing strategy in which errors are not assumed to be independent between invoking and target classes, but the testing load is lessened by careful choice of test cases

Polymorphism and inheritance can greatly complicate testing. The difficulty of testing in the presence of inheritance and polymorphism should be considered when deciding whether their use is appropriate in a given context.

SAQ 9

Given n classes $A_1, \dots, A_n$, each of which uses the method `foo` implemented in m classes $C_1, \dots, C_m$ all of which are derived from parent class C , calculate how many tests will be required using the following approaches:

- the safe approach (as illustrated in Figure 3);
- the minimal approach (as illustrated in Figure 4);
- the balanced approach (as illustrated in Figure 5) to integration testing.

What does this imply for the testing load?

ANSWER.....

- $m \times n$;
- $\max(m, n)$;
- $m + n - 1$.

$m + n - 1$ is not much bigger than $\max(m, n)$, so the balanced approach might as well be used in preference to the minimal approach. However, $m \times n$ is, in general, much bigger than both $m + n - 1$ and $\max(m, n)$. So, if the safe approach were used in preference to the balanced approach, the testing load could increase dramatically.

Unit and integration testing in component-based software development

Component-based development, such as that espoused by D'Souza and Wills (1998), can dramatically reduce development times. Part of the reduction is gained as the unit testing of components is not necessary, at least if the components are trusted (that is, they have been thoroughly tested as part of another development). Integration testing can be based on the assumption that only run-time behaviour involving a trusted component which crosses a trusted/untrusted boundary needs to be tested. This is the case when, for example, an untrusted component invokes a method in a trusted component, and vice versa.

System testing

System testing consists of checking that a completed software system performs in accordance with its requirements specification. System testing forms a comprehensive testing set that checks for conscientious software system development. Because of its complexity and importance, system testing is normally organized by the quality assurance department in large software development organizations and, if possible, should be carried out by independent validation and verification teams not involved in the development of the system.

According to Beizer (1996), system testing should consist of the following generic tests:

- ▶ **user-command testing** (or **operator testing**): tests all user commands from the point of view of tolerance to syntax errors and data input errors;
- ▶ **interface and protocol testing**: if the system communicates with other systems in the outside world, tests its interaction with the communication system;
- ▶ **start-up and initialisation testing**: tests the system's ability to be started in a working hardware/software configuration (in the case where there are many combinations of hardware/software, all configurations should be system tested individually);
- ▶ **restart testing**: tests the ability of the system to recover from errors of internal state;
- ▶ **performance testing**: tests that the system meets all specified operating requirements for speed, number of concurrent users permitted, etc.;
- ▶ **stress testing**: tests that the system can operate reliably at the limits of each of its resources (for example, to stress test a web server, we may simulate the accesses of hundreds or thousands of users all at the same time to see if it can cope with the load);
- ▶ **security testing**: tests that the system does not offer opportunities to breach security.

A popular term for starting a system is 'bootstrapping', which comes from the idiom 'pulling yourself up by your bootstraps' meaning to do something by your own efforts.

These will be performed on all subsystems (classes/packages).

A *system test matrix* is an important document in system testing, as it relates classes/packages that will be executed when the various system tests are carried out. When the system tests are as Beizer suggests, a simple system test matrix could be as illustrated in Table 3.

Table 3 A system test matrix

Test	Classes/packages				
	C1	P2	P3	P4	P5
User-command		◆		◆	◆
Interface	◆	◆			◆
Start-up	◆	◆			
Restart			◆	◆	
Performance	◆		◆		
Stress		◆		◆	◆
security	◆				

An entry ◆ in the table indicates those classes/packages executed when particular tests are carried out.

One major benefit of using system test matrices is that they ensure that system testing has adequate coverage, that is, it has been performed sufficiently thoroughly. It does this by showing those classes/packages tested, and can be used to identify whether any have been missed out.

SAQ 10

Are there any situations in which system testing should be carried out by the implementers of a system?

ANSWER.....

Probably the only situation where this is appropriate is when the project team is very small. In small teams, one person might play multiple roles, perhaps all of requirements engineer, designer, implementer, tester and maintenance engineer.

Acceptance testing

The make-up of acceptance testing is, in general, a subset of system testing. The subset is chosen by the customer and the tests are performed on the completed software by quality assurance personnel, in conjunction with the customer, in the operational environment. Its purpose is to check that the user requirements have been satisfied by the final code. On the basis of the correct functioning of the tests that make up acceptance testing, the customer formally accepts the software. For this reason, acceptance testing is called a *formal* test (this distinguishes it from the other forms of testing you have studied in this section, which are *informal*). If an acceptance test fails, the customer has the right to ask the developer to correct the software and to repeat all tests on the corrected software.

Regression testing

In regression testing all or some of the previous unit, integration and system tests are conducted to test a component or collection of components that have changed, either during developmental testing or, more importantly, during maintenance subsequent to system delivery. In working out which parts of a system need to be regression tested, a **change impact analysis** should be conducted; this identifies all parts of the system that could possibly have been affected by the change. For instance, changes in the methods of a class will affect all classes that inherit from it. However, if encapsulation is used to its fullest extent, much of regression testing can be done at the class level, because if the classes behave as they did before the change was made, then the system should also behave as it did before the change was made.

SAQ 11

Why should regression testing be necessary, since, after all, the customer has accepted the product after acceptance testing?

ANSWER.....

Acceptance testing is the process of showing that the software meets the customer's requirements, not that there aren't bugs in the code. In fact, given that a system is put into use, bugs that require fixing are almost certain to be found after acceptance testing. In addition, the system will be maintained, adding and changing functionality as needed. Regression testing is therefore necessary.



Exercise 2

Which of unit, integration, system and acceptance testing are parts of validation and which are parts of verification?

Solution

Unit and integration testing concentrate on whether parts of the system perform according to their specifications, answering the verification question (have we built the system correctly?). System and, therefore, acceptance testing concentrate on showing that the customer's requirements have indeed been met, answering the validation question (have we built the right system?). Note, however, that there are no hard and fast distinctions. Unit testing can be used to demonstrate that a component satisfies a customer's requirements – thus it can be viewed as validation, and system testing can be used to demonstrate that a system operates according to specification – in which case it can be viewed as verification.

3.3 Summary of section

In this section you have studied how verification and validation can be used to check that the quality of a software product is appropriate. Testing is currently the main activity for verifying and validating a software product. Even though testing is fallible, when properly conducted it is an essential activity in the development of software. Moreover, in combination with statistical analysis, testing can provide some confidence in the quality of a delivered software product.

An alternative to the use of testing for verification is to find ways of developing software in which consistency and completeness between the inputs and outputs of a phase are guaranteed. For example, if we could automate the way in which we derived code from a contract (expressed as assertions), we might be able to make such a guarantee. In general, this type of approach, known as formal methods, is based on mathematical techniques and notations that guarantee completeness and equivalency of system representations (for example, that the code exactly implements the contracts, that all contracts are specified and that there is no other behaviour). The use of formal methods to this degree is outside the scope of this course.

4 Measuring software quality factors

Being able to measure the software quality factors that you studied in Subsection 2.2 is an important part of being able to ensure that software quality is appropriate, that is, neither too low nor too high. In this section you will study an approach to their measurement.

You will also study how to measure software complexity, as it is important in determining where to focus testing effort. Such an approach to measurement underlies the assessment of software quality factors.

4.1 Measuring quality and complexity

Today, computing is firmly established as an engineering discipline: computing systems of immense complexity are built and generally, with notable (and very well publicised) exceptions, work well. That the complex systems we build do work as well as they do is testimony to the skills of the software engineers who build them, and to the techniques that they use. However, these techniques tend to be based on individual judgement rather than purely scientific or mathematical rules.

The contributions of applicable science and mathematics in the development of software systems are increasing. Some such contributions are:

- ▶ *finite automata* (from discrete mathematics), to describe the behaviour of a system;
- ▶ *statistics*, to show when testing is sufficiently complete;
- ▶ *formal methods*, for the development of critical components – an example being the use of mathematical notations to specify behaviours unambiguously;
- ▶ *metrics*, for the measurement of quality attributes.

In this section you will study a number of examples of metrics and their use in quality measurement.

4.2 Metrics and quality measurement

The problems of making precise measurements of software quality are discussed by McCall and Cavano. They say:

The determination of quality is a key factor in every day events – wine tasting contests, sporting events, talent contests, etc. In these situations, quality is judged in the most fundamental and direct manner: side by side comparison of objects under identical conditions and with predetermined concepts. The wine may be judged according to clarity, colour, bouquet, taste, etc. However, this type of judgement is very subjective; to have any value at all, it must be made by an expert.

Subjectivity ... also [applies] to determining software quality. To help solve this problem, a more precise definition of software quality is needed as well as a way to derive quantitative measurements of software quality for objective

analysis. ... Since there is no such thing as absolute knowledge, one should not expect to measure software quality exactly, for every measurement is partially imperfect.

(McCall and Cavano, 1978)

Measuring quality

The quality of a software product can be measured by obtaining values for the eleven software quality factors (SQFs) described in Subsection 2.2. Obtaining such values is not a trivial task, as there is no obvious way of evaluating a product against these factors. What we must do is to find properties of software products which are measurable, and use these to *infer* the values for the SQFs. McCall and Cavano have defined the following **software metrics** as measurable quantities that can be used to infer the values for the SQFs:

See, again, the paper by McCall and Cavano (1978). We've updated the language a little to be more relevant to object-oriented products.

- ▶ *accuracy*: the precision with which computation and control is carried out;
- ▶ *auditability*: the ease with which conformance to standards can be checked;
- ▶ *communication commonality*: the degree to which standard interface protocols and bandwidths have been used;
- ▶ *completeness*: the degree to which full implementation of the required functionality has been achieved;
- ▶ *complexity*: the degree to which a program is difficult to understand;
- ▶ *conciseness*: the compactness of the program in terms of lines of code;
- ▶ *consistency*: the use of uniform design and documentation techniques throughout the software development process;
- ▶ *data commonality*: the use of standard classes, patterns, etc., throughout the software development process;
- ▶ *execution efficiency*: the run-time performance of the system;
- ▶ *expandability*: the degree to which architectural, class or algorithmic design can be extended;
- ▶ *generality*: the breadth of potential application of program components;
- ▶ **hardware independence**: the degree to which the software is decoupled from the hardware on which it operates;
- ▶ **instrumentation**: the degree to which the system monitors its own operation and identifies errors that do occur;
- ▶ **modularity**: the independence of classes and system components;
- ▶ **operability**: the ease of operation of the program;
- ▶ **robustness** (also called **error tolerance**): the damage that occurs when the program encounters an error;
- ▶ *security*: the availability of mechanisms that control or protect programs and data;
- ▶ **self-documentation**: the degree to which the source code of a program provides meaningful documentation;
- ▶ **system independence**: the degree to which the system is independent of non-standard programming language features, operating system characteristics, and other environmental constraints;
- ▶ *traceability*: the ability to trace a design representation or actual program component back to the system requirements;
- ▶ **training**: the degree to which the software assists in enabling new users to use the system.

The dependence of the SQFs on these metrics is shown in Table 4.

You will not be required to recall the relationships between SQFs and metrics shown in Table 4 in the examination.

Table 4 The relationships between software metrics and SQFs

Metric	SQF										
	Correctness	Reliability	Efficiency	Integrity	Usability	Maintainability	Flexibility	Testability	Portability	Reusability	Interoperability
Accuracy		◆									
Auditability				◆				◆			
Communication commonality											◆
Completeness	◆										
Complexity		◆				◆	◆	◆			
Conciseness			◆			◆	◆				
Consistency	◆	◆				◆	◆				
Data commonality											◆
Execution efficiency			◆								
Expandability							◆				
Generality							◆		◆	◆	◆
Hardware independence									◆	◆	
Instrumentation				◆		◆		◆			
Modularity		◆				◆	◆	◆	◆	◆	◆
Operability			◆		◆						
Robustness		◆									
Security				◆							
Self-documentation						◆	◆	◆	◆	◆	
System independence									◆	◆	
Traceability	◆										
Training					◆						

The measurement of these metrics varies in difficulty. Some, such as complexity, can be directly estimated from the code of an implementation or from a UML diagram (you will see how, later in this section). Others, such as training, are subjective and require questionnaires or checklists to be developed and assessed, and calculations made therefrom.

However, given that measurements are made, the following formula can be used to estimate a value, F_q , for each software quality factor q :

$$F_q = w_1 \times v_1 + w_2 \times v_2 + \dots + w_n \times v_n$$

where $v_1, v_2, \dots, v_n$ are the measured values for the metrics which relate to q , and $w_1, w_2, \dots, w_n$ are weightings indicating the relative importance of the metrics with respect to q . In general, different software projects will require different collections of weightings, depending on the product, the customer, etc.

Example 4

Table 4 indicates that reliability is related to accuracy, complexity, consistency, modularity and robustness. Measurements of these five metrics can therefore be used to measure reliability. The formula for calculating reliability would thus be:

$$F_{\text{reliability}} = w_{\text{accuracy}} \times v_{\text{accuracy}} + w_{\text{complexity}} \times v_{\text{complexity}} + w_{\text{consistency}} \times v_{\text{consistency}} + w_{\text{modularity}} \times v_{\text{modularity}} + w_{\text{robustness}} \times v_{\text{robustness}}$$

The weightings are chosen to reflect the relative importance of the metrics in the *measurement of reliability*.

For a different SQF, the weightings would be different.

SAQ 12

From Table 4 you can see that complexity is related to each of the SQFs reliability, maintainability, flexibility and testability. Suggest why this should be the case.

ANSWER.....

Reliability is directly related to the number of bugs in a system, and the number of bugs in a system, as we have seen, is related to its complexity. Modifying complex code is likely to be more difficult than modifying simple code; maintainability and flexibility are therefore related to complexity. Determining test data for a complex system is likely to be more difficult than for a simple system; testability is therefore also related to complexity.

4.3 Measuring system complexity

There isn't room in this unit to consider in detail all 21 metrics listed above. Instead, we shall focus on just one metric, complexity, which we have chosen because of its importance in testing as well as in measuring quality. In particular, we shall look at some of the techniques that have been developed for the measurement of system complexity and their use in object-oriented systems. Perhaps surprisingly, although object orientation has been around for many years, the application of metrics to object-oriented systems is still not universally accepted. There are a number of reasons for this: one is that object orientation was originally 'sold' as being the be-all and end-all of building quality software, so that many felt that quality was automatically built into object-oriented systems. Of course, this is just hype – quality is by no means automatic in object-oriented systems. The most that can be said is that it is simpler – but still extremely difficult – to build quality systems using an object-oriented approach.

In this unit, complexity means *structural complexity*, how complicated the program code is, and not *execution complexity*, how much time/memory is required to run the program.

Lines-of-code metric

A very simple measure of the complexity of a system description is given by counting the number of lines in the description. When the system description is a piece of code, the metric produced is called the **lines-of-code (LOC) metric**.

Traditionally, the LOC metric has been claimed to provide accurate measurements of complexity. The justification is that, if we think that errors are introduced into a program randomly, then more lines of code means more errors. However, there are other factors working against it as an accurate measure: for instance, should comment lines be included in the line count? In some situations, the LOC metric does perform well: Curtis, Sheppard and Milliman (1979) have found that the LOC metric is a useful predictor of the number of errors for small units of code (in the body of a short method, for instance); Feuer and Fowlkes (1979) suggest that the LOC metric is a good predictor of maintenance effort; the COCOMO method that you will study in *Unit 13* uses the LOC metric for estimating person-years of effort in system development.

McCabe's cyclomatic-complexity metric

Tom McCabe (1976) introduced the very popular, and still widely used, **cyclomatic-complexity metric**. Since 1976, the cyclomatic-complexity metric has been used to measure the complexity of software systems developed in non-object-oriented languages such as FORTRAN, Pascal and C. It has been found to be applicable to object-oriented languages such as Java and C++.

McCabe's cyclomatic-complexity metric measures the complexity of a method by counting the *number of independent paths through a method body*. A *path* is a trace of the statements in the method actually executed, given some input conditions and ignoring all iterations except the first. Two paths are *independent* if they differ in at least one statement or expression. Consider the following graph that represents a program, where the letters *a* through *l* represent statements in the program:

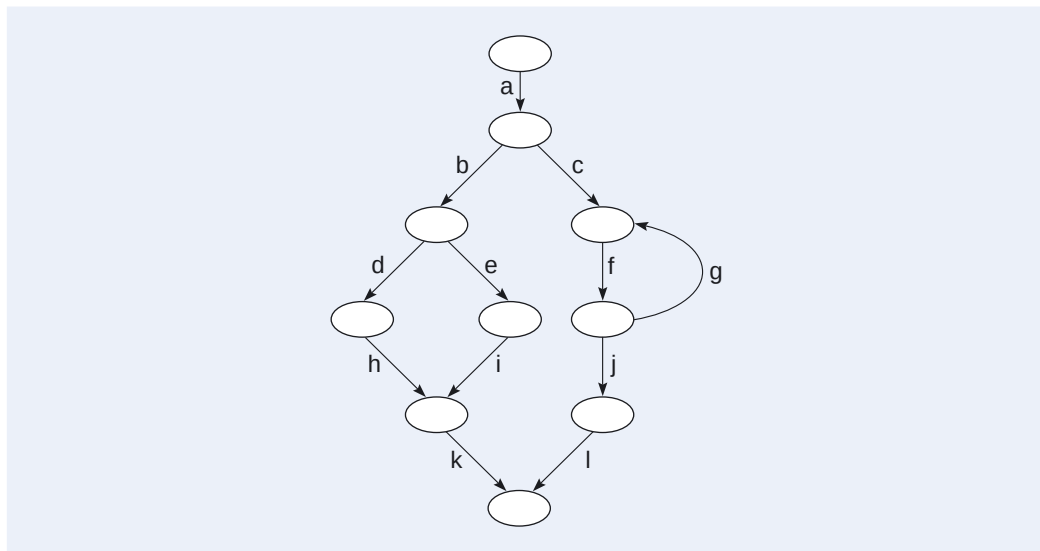


Figure 6 Computing independent paths

To compute the metric, we find the number of independent paths, taking into consideration the branches (ovals with more than one arrow exiting them). In the above example, there are 4: *abdhk*, *abeik*, *acfgjl*, *acfgjl*.

Although counting the paths might sound daunting, there is a simple way to calculate the number of independent paths through a piece of code – by counting the number of decision points. A decision point is where a choice can be made during execution; this gives rise to different paths through the code. Decision points arise through `if` statements and through `while`, `do-while` and `for` loops. A single `switch` or `try` statement can also add many more decision points.

There are tools available that when given some source code, will calculate the cyclomatic-complexity metric.

Suppose you are given a method body, and are asked to calculate its cyclomatic complexity. Cyclomatic complexity is always counted from 1: given that a method body starts and completes its execution, this defines one independent path through it (often called the *straight-through path*). An `if` statement makes a decision based on the evaluation of its Boolean condition. It can, therefore, divert from the straight-through path, and so each `if` statement in the method body adds one to the cyclomatic complexity. Each `while`, `do-while` and `for` loop adds a single decision point. Each `switch` statement adds one decision point for each of the cases it tests for, excluding any default case (we can consider a default case to be on the straight-through path). Each `try` statement adds one decision point per `catch` block, but any `finally` block is not a decision point, as it will always be executed.

In Java, the logical ‘and’ operator (`&&`) and the logical ‘or’ operator (`||`) use short-circuit evaluation. This means that they only evaluate their second operand if it is needed to determine their result. For example, if the first operand of the `||` operator evaluates to `true`, we know that the overall result will be `true` regardless of the value of the second operand. For this reason, the operators `&&` and `||` each add a decision point. The relevant decision is whether or not we need to evaluate the second operand.

The cyclomatic-complexity metric has been found to be an accurate predictor of the number of errors in method bodies; the higher the complexity as predicted by the cyclomatic-complexity metric, the higher the likelihood of errors. As a simple rule of thumb, a cyclomatic complexity of ten or more is a hint that a method body might beneficially be restructured into two or more simpler methods (if appropriate to do so).

SAQ 13

Compare the complexities of the following two pieces of code using the LOC and cyclomatic-complexity metrics. What conclusions can you draw about the relative complexity of the code?

Code A

```
int i = 1;
while (i <= 5 ) {
    playACard(i);
    if (playerHasWon(i))
        break;
    i++
}
```

Code B

```
int j = 0;
int i = 2;
j = i;
j = j + i;
j++;
System.out.println (j);
System.out.println (i);
```

ANSWER.....

Each piece of code has 7 lines. The complexity according to the LOC metric is therefore the same. Code A has cyclomatic complexity 3, whereas code B has cyclomatic complexity 1, which suggests that code A is the more complex of the two. This supports an intuitive view that the structure of code A appears more complex than that of code B.

Object-oriented metrics

It is generally accepted that object-oriented systems require two levels of complexity metric: one to measure method complexity, the other to measure the complexity of the class structure. In fact, because the structuring mechanism for object-oriented systems – inheritance – is so radically different from that of procedural languages, new complexity metrics have had to be developed.

Chidamber and Kemerer (1994) are in the vanguard of this, and have provided a suite of complexity metrics for use specifically with object-oriented systems, which mainly measure structural complexity in terms of the interaction of classes and inheritance. These metrics are as follows.

- ▶ **Depth-of-inheritance-tree (DIT) metric:** for a given class, DIT is defined as the largest number of hops through an object's superclasses, where the starting class is numbered 0. For a single inheritance programming language like Java, this means that the DIT is the number of ancestors + 1, but this is not true in languages like C++ that support multiple inheritance.
A method in the given class can be defined in that class or in any of its ancestors. DIT is, therefore, a measure of the number of classes essential to the understanding of the methods of the given class. The deeper a class is in an inheritance tree, and so the higher the value of this metric, the more difficult the class may be to understand.
- ▶ **Coupling-between-objects (CBO) metric:** for a given class, CBO is defined as the number of relationships the class has with other classes. The more relationships a class has, and so the higher the value of this metric, the more difficult it is to understand the use of the given class.
- ▶ **Number-of-children (NOC) metric:** for a given class, NOC is defined as the number of immediate children for that class. This metric is a measure of the number of classes that will be affected by changes to a given parent class.
- ▶ **Response-for-a-class (RFC) metric:** for a given class, RFC is defined as the size of the **response set** for the class, which consists of all the methods of this class (including methods inherited from superclasses), together with all the methods that are invoked on objects of other classes. Like DIT and CBO, this metric measures the number of methods needed to understand a class. Classes with high values for this metric will be more difficult to understand than classes with low values.
- ▶ **Lack-of-cohesion-in-methods (LCOM) metric:** for a given class, LCOM measures its **cohesiveness**. Ideally, a class should represent a single domain concept, reflected by the fact that the attributes of a class should be closely related. LCOM is defined as the number of pairs of methods that do not make reference to the same attributes, minus the number of pairs of methods that do, or zero should this be negative. In highly cohesive classes, methods will manipulate the same attributes.
- ▶ **Weighted-methods-per-class (WMPC) metric:** given a class, WMPC measures its complexity of behaviour. It is defined as the sum of the cyclomatic complexities of each method of the class. The relationship between this metric and complexity is that a class with many simple methods may be as difficult to understand as a class with a few complicated methods.

Chidamber and Kemerer have provided evidence that classes with CBO values greater than 7 or RFC values greater than about 50, account for the majority of problems in object-oriented systems.

In Java, the DIT of the class Object, the superclass of all other objects, is 0, so the minimum DIT of a user class is 1.

Interface classes are generally given a depth of 1, even if they extend another interface class.

Some people and tools count constructors as methods, and others don't. We recommend that they be counted. Do not count catch clauses, even though they look and behave somewhat like methods.

One advantage of Chidamber and Kemerer's metrics is that, other than the WMPC metric, they can usefully be applied very early in the software development process. For instance, we can take a UML class diagram and apply the CBO metric to measure its complexity. This is a very promising use of object-oriented metrics, and means that object-oriented system specifiers and designers can feed complexity information into the development process very early on.

SAQ 14

Look again at the definition of the DIT metric. Do you think that, in calculating the metric, classes from the Java API should be included? Do you think that API classes should be counted in Chidamber and Kemerer's other metrics?

ANSWER.....

Chidamber and Kemerer's metrics measure complexity, and API classes add to complexity. Hence they should be counted in *all* metric calculations. For this reason, the Java documentation will be very useful when calculating these metrics.

SAQ 15

The WMPC metric measures the complexity of a class in terms of the sum of the cyclomatic complexities of its methods. For a class with 10 methods, suggest a WMPC value above which the behaviour of the class should be considered too complex.

ANSWER.....

For individual methods, a cyclomatic complexity of greater than 10 should be regarded as a hint that the method is too complex. For a class with 10 methods, therefore, a value for the WMPC metric of greater than $10 \times 10 = 100$ should be regarded as indicating that the behaviour of the class is too complex.

Exercise 3

Should the cyclomatic-complexity metric be used to measure the complexity of a whole software system?

Solution

Because the cyclomatic-complexity metric is based on decision points, which are present only in methods, it is 'blind' to the class-structuring mechanisms that are available in object-oriented system descriptions. As much of the complexity of an object-oriented system is held in the class structure, applying the cyclomatic-complexity metric to a whole system would not, therefore, be appropriate.



Exercise 4

The following piece of code is part of `TherapyServer` (based on an idea from Budd [2000]), an unfaithful translation into Java of Eliza, a program that parodied a therapist to demonstrate some artificial intelligence and natural language processing concepts. Calculate the cyclomatic complexity for this method by filling in the right-hand column, cumulatively.



Eliza was created in 1966 by Joseph Weizenbaum at MIT.

Table 5 Code from *TherapyServer*

Line number	Code	Cyclomatic complexity
1	<code>public String response (String text) {</code>	
2	<code> if (text.endsWith("?"))</code>	
3	<code> return "Why do you want to know?";</code>	
4	<code> Vector words = new Vector();</code>	
5	<code> StringTokenizer breaker = new</code> <code> StringTokenizer(text.toLowerCase(),</code> <code> " .,?!");</code>	
6	<code> While (breaker.hasMoreElements())</code>	
7	<code> words.addElement(breaker.nextElement());</code>	
8	<code> if (words.size() > 1 &&</code>	
9	<code> words.elementAt(0).equals("i") &&</code>	
10	<code> words.elementAt(1).equals("feel"))</code>	
11	<code> return "So you feel " +</code> <code> words.elementAt(2);</code>	
12	<code> for (int i = 0; i < words.size(); i++) {</code>	
13	<code> String relative =</code> <code> (String) words.elementAt(i);</code>	
14	<code> if (isRelative (relative))</code>	
15	<code> return "Tell me more about your " +</code> <code> relative;</code>	
16	<code> }</code>	
18	<code> return "Tell me more";</code>	
19	<code>}</code>	

Solution.....

The table should have the following entries, giving a cyclomatic complexity of 8.

Table 6 Calculation of cyclomatic complexity

Line number	Cyclomatic complexity
1	1 (starts at one)
2	2 (if statement)
6	3 (while statement)
8	4, 5 (if statement, &&)
9	6 (&&)
12	7 (for loop)
14	8 (if statement)

4.4 Summary of section

In this section you have studied the relationship between software quality factors and software metrics. You have seen that software quality factors are not generally directly measurable, but that McCall and Cavano have introduced 21 software metrics which are directly measurable, and which can be used to estimate software quality factors. You have studied how one of the software metrics, complexity, is measured through lines of code, cyclomatic complexity and a variety of object-oriented measures.

5

Testing techniques

Beizer (1990) contains a complete, and accessible, discussion of software testing.

As we stated in Subsection 3.2, testing is one of the most important activities undertaken by a software engineer when it comes to validation and verification. Used correctly, testing is a useful way to guard against the human tendency to introduce errors when developing software. The techniques used for testing are many and various. In this section, we shall describe the two main categories of testing techniques – black box testing and white box testing – and show how they can be applied to code.

5.1 Black box and white box testing

The essential difference between our two categories of testing technique is the way in which tests are designed:

- ▶ in **black box testing** we design test cases by looking at the specification (that is, requirements and high-level design) of the system to be tested;
- ▶ in **white box testing** we design test cases by looking at the detail of the implementation of the system to be tested.

Because of the ways in which test cases are designed, black box and white box testing have complementary roles in the testing process; a balanced testing strategy will contain elements of both black box and white box testing.

- ▶ Black box testing is used to test that each aspect of the customer's requirements is handled correctly by an implementation. Black box testing 'sees' a system through its specification – the specification defines what the system should do and, through black box testing, we make sure it does it.
- ▶ White box testing is used to check that the details of the implementation are correct. White box testing ignores the 'big picture' of the requirements, and instead looks to detailed designs to check that the system does what it is supposed to do, correctly.

Both black box and white box testing were developed originally to test software systems built using procedural languages. The behaviour of object-oriented systems is structured around objects, with methods whose behaviour depends critically upon the states of objects; some modern authors claim that this behaviour differs from the simple input/output model which forms the basis of black box testing, and this reduces the adequacy of black box techniques for testing object-oriented systems. This viewpoint reinforces the observation that a balanced testing strategy should include both black box and white box testing.

Black box testing techniques

The techniques that can be used in black box testing are characterised by the fact that they can use only information available from the specification in order to develop test cases. To produce test data for a (sub)system, therefore, only the defined relationships between inputs and outputs can be scrutinised.

By 'relationships between inputs and outputs' we mean either their functional relationships (that is, how one is turned into the other) or the efficiency with which outputs are produced.

The quintessential black box technique is **partitioning** (also known as *equivalence partitioning*) combined with *boundary testing* (also known as *fence testing*), which focuses on producing test data at the boundaries between *partitions* of the input data space (or input domain). The input data space is partitioned into subdomains, where a *subdomain* is a set of input values that require the same type of processing to be performed. Subdomains are obtained by the technique of *case analysis*, which determines for each user-perceived function of the (sub)system, the subdomain which results in that function being performed. Boundary testing is based on the observation that common errors are often caused by a data item being just one out, or, for example, a loop being executed one too many or one too few times; such errors are most visible at the boundaries of the input data space.

The *input data space* is the set of all values that method arguments can take.

An example of partitioning into domains using case analysis is given in Example 5.

If we have designed our software using defensive programming, we will need to produce partitions that cover all of the possible values that our arguments can take. For example, to test a (static) method to compute the square root of an integer, we might partition the input space into two sets: integers less than zero, and integers greater than or equal to zero. The idea here being that we need to check that our code does something sensible with negative values – for example, produce an error report, as well as taking the square root of non-negative values. Under Design by Contract (DbC), we need to test methods only for values that are valid according to the method's pre-conditions. For example, under Design by Contract our square root method might expect the client to filter out negative values. We will then need only to test for values greater than or equal to zero.

In **random testing**, test data is randomly generated within the input data space. Random testing is very good at generating data a human tester would not think of. Moreover, it is cheap in terms of tools support. Its main disadvantage is that, for large programs, the amount of test data that needs to be generated can be very large.

In **error guessing**, the most unexpected (or, perhaps, the most bizarre) test data that can be thought of are presented to the program unit. Some people have developed an intuitive flair for creating test data that reveals errors. Although this technique is informal, it cannot be ignored – it has achieved considerable success in the past!

A strategy for black box testing (using partitioning)

We assume that we have been given a class (package, or other subsystem) to test and that the basic operations for the class have been determined from the use cases. Test data needs to be chosen to show that the class performs those operations according to the use cases.

Step 1: For each method in the class, determine the input data space

Using the use cases or other parts of the UML system description, the input data space for each method of the class can be determined.

Example 5 (part 1)

In an airline reservation system, a `Booking` class allows an operator to update a particular booking using an `update` method that has two inputs – `command` and `flightNumber` – if a summary of the flight is required, and three inputs – `command`, `flightNumber` and `seatNumber` – if reservation or cancellation of a seat is required. The inputs are as follows:

- ▶ `command` – one of `reserve`, `cancel` or `summary`;
- ▶ `flightNumber` – an integer in the range 1 to 999;
- ▶ `seatNumber` – an integer in the range 1 to 450.

The input data space for the `update` method thus consists of all possible combinations of three values taken from $\{\text{reserve}, \text{cancel}\}$, $\{1, 2, \dots, 999\}$ and $\{1, 2, \dots, 450\}$ together with all possible combinations of two values taken from $\{\text{summary}\}$ and $\{1, 2, \dots, 999\}$.

Step 2: Partition the input data space into subdomains

Once the input data spaces for the class's methods have been determined, they must be partitioned into a set of subdomains using case analysis. Our partitioning has assumed that our software has been designed and implemented using a Design by Contract approach, meaning that the caller is responsible for not passing out-of-range data.

Example 5 (part 2)

When the `reserve` command is entered together with a `seatNumber` and a `flightNumber`, a seat is reserved. When the `cancel` command is entered together with a `seatNumber` and a `flightNumber`, a seat reservation is cancelled. When the `summary` command is entered together with a `flightNumber`, the total number of seats booked is displayed. These three cases correspond to three distinct user-perceived functions of the airline reservation system and lead naturally to a partition of the input data space of the `update` method into three subdomains, as shown in Table 7.

Table 7 The user-perceived functions for the `update` method

User-perceived function	Subdomain
Reserve a seat	<code>command = reserve</code> $1 \leq \text{flightNumber} \leq 999$ $1 \leq \text{seatNumber} \leq 450$
Cancel a seat	<code>command = cancel</code> $1 \leq \text{flightNumber} \leq 999$ $1 \leq \text{seatNumber} \leq 450$
Summarise seats booked	<code>command = summary</code> $1 \leq \text{flightNumber} \leq 999$

Thus, for example, the ‘reserve a seat’ user-perceived function is exercised by messages of the form (for an object of class `Booking`, `aBooking` say)

```
aBooking.update(reserve, flightNumber, seatNumber)
```

where `flightNumber` is between 1 and 999 inclusive and `seatNumber` is between 1 and 450 inclusive.

Step 3: Test all subdomains given by the case analysis

Once the input data space has been partitioned into subdomains by case analysis, black box testing can take place by choosing test data for each subdomain. This test data should include not only typical (or expected) data in the middle of each subdomain, but also at the extremes of the subdomain and values close to the extremes (to detect errors that may arise if the programmer has used the wrong comparison operator).

Example 5 (part 3)

Table 8 shows suitable test data for the summary command.

Table 8 Test data for the `summary` command

Command	Flight number
summary	1
summary	2
summary	500
summary	998
summary	999

The extreme values for testing the functionality of the `summary` command are 1 and 999, so we test for the correct processing of these values, along with 2 and 998 (which are values near the extremes) and for 500, the value in the middle of the subdomain.

One major advantage of the three-step black box testing strategy described above is that it allows all possible user-perceived functions to be tested. However, for many (sub)systems the number of subdomains can be enormous, and hence the effort involved in testing them all can be so large as to be prohibitive.

SAQ 16

Use the strategy for black box testing described above to choose a good set of test data for the cancel command discussed in Example 5 (part 2).

ANSWER.....

The subdomain for the `cancel` command is shown in Table 9. We need to test for each `flightNumber` in Table 8, and for each such `flightNumber` we need to test extreme, near-extreme, and central values of the `seatNumber` data. Five suitable test values for the `seatNumber` data are 1, 2, 200, 449 and 450. Thus a suitable set of test data for the `cancel` command is shown in Table 9.

Table 9 Test data for the `cancel` command

command	flight Number	seat Number	command	flight Number	seat Number	command	flight Number	seat Number
cancel	1	1	cancel	500	1	cancel	999	1
cancel	1	2	cancel	500	2	cancel	999	2
cancel	1	200	cancel	500	200	cancel	999	200
cancel	1	449	cancel	500	449	cancel	999	449
cancel	1	450	cancel	500	450	cancel	999	450
cancel	2	1	cancel	998	1			
cancel	2	2	cancel	998	2			
cancel	2	200	cancel	998	200			
cancel	2	449	cancel	998	449			
cancel	2	450	cancel	998	450			

White box testing techniques

White box testing can be used to concentrate on well known sources of developmental errors. For instance, logical errors on the part of the programmer are more likely in code which is not often executed, for example, in code which deals with special cases or initialization methods. Here we consider just two white box testing techniques:

basis-path testing and **loop testing**.

Basis-path testing

Basis-path testing was developed originally by Tom McCabe, and is based on his cyclomatic-complexity metric (which you studied in Subsection 4.3). It ensures that *all* reachable statements in a method are tested at least once. You will remember that the cyclomatic-complexity metric counts the number of independent paths through a method body. It therefore provides us with an indication of the *maximum* number of tests that need to be performed in order to traverse paths such that each reachable statement and branch in a program is executed at least once. (It might be that fewer tests could achieve this, for example, two `if` statements might be controlled by the same condition.) Of course, testing each path once may not be sufficient. One reason for requiring more tests is that we want to test more thoroughly the statements in several independent paths because of the interactions within the method, such as an `if` statement sometimes modifying a variable used later in the program, perhaps in another `if` or in a `for` loop. The selection of the paths to test is driven both by coverage (the basis paths) and by the details of the method itself (internal interactions).

One way to determine basis-path test cases is as follows: first select data that will exercise the *straight-through path*, that is, all loop and `if` conditions evaluate to `false`, and only default cases are selected in `switch` statements. Then, find data that deviates from the straight-through path at the first decision point. The process is then repeated for each subsequent decision point in the program, varying the flow in each case. The following example illustrates the technique.

A method might have unreachable statements if some combination of conditions can never be satisfied. This typically happens when code is added to deal with errors that are violations of the method's preconditions, or are violations of postconditions of the methods it uses.

Note that it might not be possible to construct test data such that all conditions evaluate to false. For example, the method might contain both

```
If (a) ...
and
if (!a) ...
```

In this case you treat each mutually-exclusive path as a straight-through path.

Example 6

In Exercise 4, you were asked to calculate the cyclomatic complexity of the response method of the `TherapyServer`. The method was found to have a cyclomatic complexity of eight, meaning that there are eight independent paths through the code. Basis-path testing for this method will, therefore, have no more than eight test cases. These are generated by analysing the decision points and constructing test cases to exercise each branch. Table 10 shows the seven branch points (the straight-through case gives the eighth test case), together with test data that will exercise each of them.

Table 10 The seven branch points of the response method of the `TherapyServer`

Line Number	Code	Straight-through case	Branch-taken case	
2	<code>if (text.endsWith ("?"))</code>	" " (empty string)	"Hello?"	
7	<code>while(breaker.hasMoreElements())</code>	" " (empty string)	"Hello"	
9	<code>if ((words.size() > 1) && words.elementAt(0).equals("i") && words.elementAt(1).equals("feel"))</code>	"Hello"	"Hello there"	
9	<code>if ((words.size() > 1) && words.elementAt(0).equals("i") && words.elementAt(1).equals("feel"))</code>	"Hello there"	"I worry"	
10	<code>if ((words.size() > 1) && words.elementAt(0).equals("i") && words.elementAt(1).equals("feel"))</code>	"I worry"	"I feel unhappy and lonely"	
13	<code>for (int i=0; i<words.size(); i++)</code> {	" " (empty string)	"I worry"	
15	<code>if (isRelative(relative))</code>	not tested	straight-through case	branch-taken case
			"I worry"	"Hello mother"

The straight-through case for the Boolean operators is the case in which they do not have to evaluate their second operand. We have underlined the relevant operator (the operator where evaluation stops) in the above table entries.

Notice that, although we earlier calculated a cyclomatic complexity of eight, we only need seven test cases. This is because the test data identified for the decision point in line 13 has already been introduced for earlier decision points. Notice also, that the last `if` statement isn't tested by the

main straight-through case, because it is nested inside the `for` loop. Therefore we've split the branch-taken case for the enclosing `for` loop into two in order to test it.

The following code provides a test harness for the `TherapySession` class that was used to run the `response` method with the test data above:

```
import java.io.*;
// response () uses Vectors
import java.util.Vector;

// response () uses StringTokenizer
import java.util.StringTokenizer;

public class TestHarness {

    static public void main (String [ ] args) {
        TherapyServer freud = new TherapyServer();
        System.out.println ("Test case: *" + " "
            + "** Result: *" + freud.response (" ")
            + "**");
        System.out.println ("Test case: *" + "Hello?"
            + "** Result: *" + freud.response ("Hello?")
            + "**");
        System.out.println ("Test case: *" + "Hello"
            + "** Result: *" + freud.response ("Hello")
            + "**");
        System.out.println ("Test case: *" + "Hello there"
            + "** Result: *" + freud.response ("Hello there")
            + "**");
        System.out.println ("Test case: *" + "I worry"
            + "** Result: *" + freud.response ("I worry")
            + "**");
        System.out.println ("Test case: *"
            + "I feel unhappy and lonely"
            + "** Result: *"
            + freud.response ("I feel unhappy and lonely")
            + "**");
        System.out.println ("Test case: *" + "Hello mother"
            + "** Result: *" + freud.response ("Hello mother")
            + "**");
    }
}

public class TherapyServer {
    // constructor, etc
    public String response (String text) {...}
    ... // other methods used by the response method
}
```

The results of the execution of the test harness are:

```
Test case: ** Result: *Tell me more*
Test case: *Hello?* Result: *Why do you want to know?*
Test case: *Hello* Result: *Tell me more*
Test case: *Hello there* Result: *Tell me more*
Test case: *I worry* Result: *Tell me more*
Test case: *I feel unhappy and lonely* Result: *So you feel unhappy*
Test case: *Hello mother* Result: *Tell me more about your mother*
```

Loop testing

Loops create some difficulties for testing; they can, after all, end up being executed endlessly. Loop testing is a white box testing technique that is used to test loop constructs. In Java, there are only three ways in which loops can occur: *simple loops*, *nested loops* and *concatenated loops*. Each has a different testing strategy.

These loop testing strategies are adapted from Beizer (1990).

In Java, simple loops have one of the following forms:

```
for (initial;
     test;
     increment)
loop body;

while (condition)
loop body;

do
loop body;
while (condition);

for (array or
     collection expr)
loop body;
```

For each of these, test data should be found for the following cases:

- ▶ the loop is skipped entirely (for example, the entry condition for a `while` loop is `false`; this is not possible for the `do-while` loop, which simplifies its testing slightly);
- ▶ the loop is passed through exactly once;
- ▶ the loop is passed through more than once.

If, in addition, the body of the loop is expected to be executed a maximum of n times, (which might happen if looping over input data that came from a form field with a fixed size of n , or if producing output that is expected to fit into n columns), then test data should be found for cases which attempt:

- ▶ to pass through the loop m times, where m is chosen to be in the range 2 to $n - 2$;
- ▶ to pass through the loop $n - 1$, n and $n + 1$ times. The last value, $n + 1$, tests what happens if assumptions are violated. This test is not required if the design contract specifies a maximum of n as an initial condition.

Assumptions about lengths of input fields and widths of output fields may be specified in a contract. Unfortunately, these assumptions are often left unstated.

Nested loops are loops which have, within their body, other loops. For example:

```
for (int i = 0; i < 10; i++ )
  for (int j = 0; j < 20; j++ )
    for (int k = 0; k < 30; k++ )
      loop body;
```

Nested loops, if tested using the strategy for simple loops, would generate unmanageable amounts of test data. To cut down the number of cases, the following approach can be used.

- 1 Start with the innermost loop, set all other loop counter variables to their minimum values (in the case of the example, set $i = 0$ and $j = 0$ and change the limits of the outer and middle loops to $i < 1$ and $j < 1$, respectively) and conduct a simple loop test on the whole loop.
- 2 Work outwards, and conduct a simple loop test on the next outermost loop, while keeping all outer loops to 'typical' values (for the example, retain $i = 0$ and the limit on i of $i < 1$).
- 3 Repeat step 2 until all loops are tested.

Concatenated loops are loops that follow each other. An example of concatenated loops is:

```
while (i < 10)
    loop body;
while (j < 20)
    loop body;
```

Most concatenated loops can be tested separately using sequential simple loop tests. However, look at the case where a later loop uses the termination conditions of an earlier one as its starting values, as for example in:

```
while (i < 10)
    total = total + i;
// the next loop uses total, which is set by
// the previous loop
while (j < total)
    System.out.println (j);
```

The situation is then slightly more complex, and the nested loop testing strategy should be performed instead.

SAQ 17

What is the point of testing the case $n + 1$ for a loop that is intended to execute no more than n times?

ANSWER.....

To ensure that the termination condition of the loop is correct.

5.2 Ensuring testing is adequate

Problems with black box testing

A large amount of data has been collected which suggests that black box testing, used in isolation, suffers major drawbacks. In one study, black box testing was performed on over 40 000 lines of program code from a real software project. The study measured the proportions of individual statements that were exercised by the black box testing. The results from this study are shown in Figure 7. In all, fewer than half (44.5 per cent) of the simple assignment statements and only around a third (35.1 per cent) of the decision points were tested. The results indicate poor coverage levels when a black box testing strategy alone is pursued.

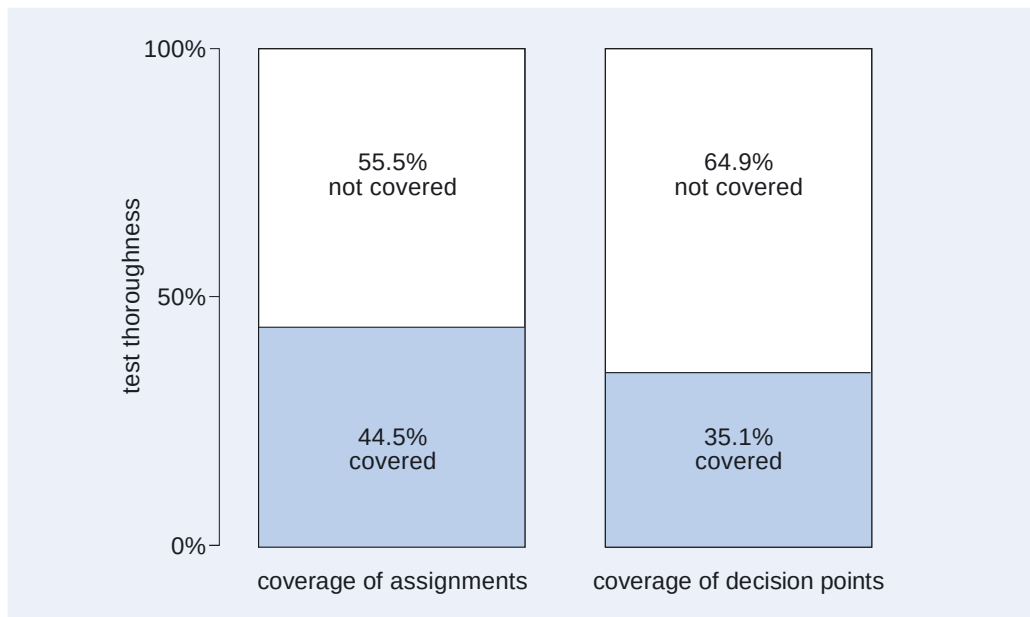


Figure 7 The thoroughness of black box testing

Moreover, although the process of generating test data for black box testing is straightforward, a major inadequacy of black box testing is revealed if a tested method's operation depends for its behaviour on the internal state of an object. As an example suppose that the `Booking` class (in Example 5) includes an attribute `available` to indicate whether a seat is available or not, but that this attribute is not visible through the interface to the method (and so will not be known of by a black box tester). The behaviour of the `reserve` method should be different when the seat is already reserved (and so `available` is `false`) from when it is not. In black box testing, however, we must ignore the internal detail of the class, and so any inconsistencies between the values of `available` and other aspects of the objects would necessarily be missed.

There are two ways around this inadequacy. The first way can create a great deal of extra work for the tester: not only must case analysis be done for each command, but all possible sequences of commands should be tested. For instance, the sequence `reserve a seat, cancel the same seat` should be run, as should `reserve a seat, cancel a different seat`. Clearly, the number of test cases increases dramatically if this testing strategy is used. The second way, which is easier, is always to conduct black box testing in tandem with white box testing.

Problems with white box testing

White box testing is not without its own problems. The main one is that white box testing alone, by concentrating on the code of an implementation, may not reveal customer requirements that have been omitted in an implementation. The solution is, again, to combine black box and white box testing.

Another problem with *both* black box and white box testing is that, for comprehensive testing of large systems, immense quantities of test data are required. Often the only way to succeed in managing testing is to use automated tools. Many aspects of testing become routine when using tools, whereas they were generally considered a chore before tools existed. For instance, a tool can easily and automatically do a detailed change analysis, so that much of regression testing can be automated.



Exercise 5

Give a situation in which black box testing will test something that white box testing would miss, and one in which white box testing will test something that black box testing would miss.

Solution

Because black box testing takes its test cases from the specification, it is likely to pick up the following sorts of errors that white box testing would miss (not an exhaustive list):

- ▶ operations required by the specification but not provided for by the implementation;
- ▶ errors between the interfaces of two classes;
- ▶ errors in the transformations between internal states of a class, if these affect input/output behaviour;
- ▶ performance errors, in which system performance is found to be wanting;
- ▶ system initialisation and termination errors.

On the other hand, in looking inside the implementation, white box testing will pick up the following sorts of errors that black box testing would miss (again, not an exhaustive list):

- ▶ that the sequences of method calls in the body of a method perform the correct function;
- ▶ that Boolean conditions in `if` statements, `while` loops, etc. are correctly expressed;
- ▶ that loops terminate correctly;
- ▶ that the relationships between methods and attributes of classes are correct.



Exercise 6

A weather-recording system records wind-speed data, rainfall data and barometric pressure, and sends summary data into a computer network. If the wind-speed data is represented as an integer between 0 and 110, the rainfall data is represented as a floating point number between 0.0 and 150.0 (significant to 1 decimal place) and barometric pressures is represented as an integer between 800 and 1200, what is the input data space?

Solution

The input data space is the set of triples of values taken from the sets $\{0, 1, \dots, 110\}$, $\{0.0, 0.1, \dots, 150.0\}$ and $\{800, 801, \dots, 1200\}$.



Exercise 7

Do the test data for the `TherapyServer` (in Example 6) reveal any deficiencies in the code?

Solution

Whether an answer from `response` is a deficiency depends upon what the requirements for the `TherapyServer` are, and we were not given these. Thus we cannot say if it is correct or not.



Exercise 8

Develop test data for the simple `while` loop that appears at line 7 of the `response` method of `TherapyServer` code (see Table 10).

Solution

" " (the empty string) will drop straight through the loop.

"hello?" will force the loop to execute once.

"hello there" will force the loop to execute twice.

As there is no a priori bound on the number of times that the loop can execute, only the first three test cases for simple loops apply.

5.3 Summary of section

In this section you have studied the two main techniques for software testing, black box and white box testing. Black box testing provides evidence that a software system meets its requirements, whereas white box testing provides evidence that the detail of the implementation is correct. Both black box and white box testing have problems individually, especially in testing object-oriented systems, and a balanced testing strategy will include aspects of them both.

6

Formal technical reviews

Formal technical reviews, or *inspections* as we prefer to call them, provide a holistic solution to the problem of ensuring the quality of a software product. Inspections can be – and have been – used on all products of the software development process. In fact, inspections have been used throughout the whole of business process engineering: ‘whole company’ objectives – business plans, sales plans, etc. – can be put through the inspection process and benefit from it.

Our focus in this section will be on the application of inspections to improving product quality. However, there are many different ways in which inspections are used in the software development process, and we do not have room to discuss them all here. Instead, we shall focus on general principles and techniques.

6.1 Inspections

The purpose of an inspection is to critique a product of the software development process with a view to improving it. The product might be a customer requirements document, a piece of code or a test or maintenance plan.

Michael Fagan is credited with the original work on inspections (another name for inspections being *Fagan inspections*). Fagan (1976) states that, ‘inspections are a formal, efficient, and economical method of finding errors in design and code’. Fagan applied inspections to three types of product as part of the study on which his paper was based: the design, the pre-tested code and the post-tested code. He concluded that inspections of the design and (to a lesser extent) pre-tested code were very worthwhile, producing large productivity gains by finding bugs (or defects) early in the development process. Inspecting post-tested code was found to be useful in finding defects, but not in a cost-effective way: the time could have been better spent testing.

Inspections improve products by finding defects: figures quoted vary widely, but it is claimed that 60 to 100 per cent of defects can be found through the use of inspections (although the higher of these figures is, of course, unrealistic for large systems [Russell, 1991]). Through inspection, defects tend to be found very early in the development process, when the cost of removal is cheaper.

Inspections have useful side-effects as well.

- ▶ Inspections provide an excellent training ground for technical personnel. The participants in an inspection can gain much valuable experience of the process of conducting an inspection, of the methods that are used to produce the product, and of in-house standards. They can also cement working relationships (it is rumoured that people can get to enjoy inspections).
- ▶ Inspections provide information about the processes used to develop the product, by allowing statistics on defects throughout a single development process and over multiple development processes to be collated.
- ▶ If practised routinely, inspections can improve the process of inspecting.

Personnel

Fagan was very specific about who should be involved in an inspection, and about the skills that they should have. The following list of personnel is adapted from the *Software Formal Inspections Guidebook* – the *de facto* standard for inspections.

- ▶ **Moderator** The role of the moderator is to manage the inspection and to provide leadership. The moderator should be tactful, sensitive and able to show the drive to keep the inspection going. The moderator also has to organise and schedule the meetings, put in place the infrastructure that will allow the inspection to get going, and produce the goods with prompt reporting and following-up. The moderator should be an able programmer, but not necessarily a domain expert in the topic of the program. (Given all these talents, it is not surprising that Fagan (1976) describes the moderator as, ‘the key person in a successful inspection’.)
- ▶ **Scribe** The scribe is responsible for accurately recording, during the meetings, information about each defect found. The information should include where to find the defect, its classification, and the identity of the reviewer who found it.
- ▶ **Author** The author is the developer of the product being inspected.
- ▶ **Reviewers** The reviewers are the people who do the actual inspecting. There should be between two and five reviewers. One will be a *reader*, who is the reviewer responsible for guiding the inspection.

Much of the terminology in this section will be found in the *Software Formal Inspections Guidebook* (OSMA, 1993). It is an excellent source of information about inspections.

Defect classification is discussed below.

Processes

Fagan is very explicit that, for maximum benefit to be derived from an inspection, the product should be ready to be inspected, and that it should not emerge from the inspection process (for use by other actors in the software development process) until all stages of the inspection process have been completed. The *Software Formal Inspections Guidebook* defines both *entry* and *exit criteria* for an inspection.

- ▶ **Entry criteria** depend on the product, but in general require that the product is sufficiently mature to be used as it stands after the inspection process is complete. For software, this means that the software successfully compiles, although in-house rules for style (which can be automatically tested) might be added to the entry criteria as well.
- ▶ **Exit criteria** are usually that all major defects found have been corrected.

Given that a product satisfies the entry criteria, it can be inspected. The inspection process consists of seven stages. The stages are shown in Figure 8.

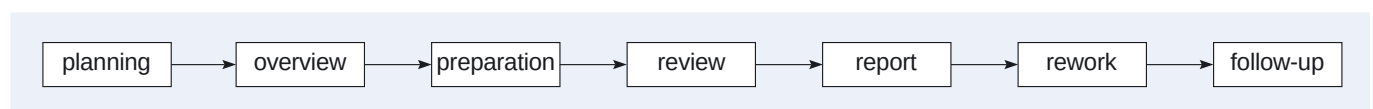


Figure 8 The seven stages of an inspection

Planning The objectives of the **planning** stage are to:

- ▶ determine what will be inspected;
- ▶ determine whether the product is ready to be inspected;
- ▶ collect supporting materials for the inspection (forms, checklists, etc.);
- ▶ determine who will be the reviewers;
- ▶ schedule the inspection process;
- ▶ help the author prepare for the next stage (the overview).

These tasks are the responsibility of the moderator.

Overview The aim of the overview stage is to allow the inspection team to become familiar with the intention and detail of the product. The objectives are:

- ▶ for the author of the product to provide an overview;
- ▶ for product and supporting materials to be distributed;
- ▶ for the scribe for the inspection to be appointed;
- ▶ to ensure that all inspectors are committed to the preparation phase.

Preparation An important contribution to the process is to prepare thoroughly for the inspection. The single objective for preparation is to find defects of the product and to classify their *importance* and *type*. This is an individual task, and vital because the review meeting (described below) has no purpose without a list of defects to discuss.

During preparation, a defect should be classified as either *major* or *minor*.

- ▶ a **major defect** is one which, if not corrected, would either cause a malfunction, or prevent the attainment of a required result;
- ▶ a **minor defect** is one which, if not corrected, would neither cause a malfunction nor prevent the attainment of a required result.

The defect should also be classified according to its type, which could be one of, for example, data, requirements compliance, interfaces, logic, standards compliance, performance, readability (though other types might also be equally appropriate).

Review The main objective of the review meeting is to confirm and list major defects, and to determine who will be responsible for addressing each defect. The review meeting should also set team objectives such as:

- ▶ developing teamworking;
- ▶ creating shared knowledge of the product and of the development process;
- ▶ improving the reviewing skills of participants.

Report Within one day of the review meeting, the scribe should produce a written report of the inspection and the major defects that were found. The main objective of the report is to record who is responsible for addressing each defect.

Rework The objective of reworking is to correct the defects listed in the report.

Follow-up The main objective of the follow-up meeting is to ensure that all major defects have been dealt with adequately. If they have not, the product is failed and the reworking stage is repeated; if they have, the product is passed, provisionally. Once passed provisionally, the follow-up meeting assesses the amount of reworking that has taken place. If it is found that more than 5 per cent of the product was reworked, the review team should reconvene and carry out a full reinspection. If less than 5 per cent of the code had been reworked then the moderator, at his or her discretion, can be left with the reinspection. Once reinspection is deemed satisfactory (several reinspections may be needed in some cases), the product is passed.

There is also what might be called a holistic objective for the follow-up meeting. This is to assess the success, or otherwise, of the inspection process. Meeting this objective ensures that the effectiveness of the inspection process itself keeps growing and ensures the investment made in it is worthwhile.

6.2 Making the most of inspections

The following 'golden rules' for inspections will ensure that the most is made of what is an expensive process.

- 1 *Inspect the product, not the producer.* Don't allow egos to be damaged by making any person its target.
- 2 *Set an agenda and stick to it.* Understanding and focusing on the purpose of the review meeting is the best way to make use of the limited time.
- 3 *Limit debate and rebuttal.* Defects raised that cause debate should be put aside for separate discussion later.
- 4 *Note defects, but don't attempt to fix them.* As soon as a defect is identified, record it and the name of the person who will be responsible for addressing it, and move on to the next. Time is precious in the review meeting, and the point is to collect defects, not to work out how to fix them.
- 5 *Take written notes to record the defects.* It's easy to forget, especially when you're dealing with many defects.
- 6 *Limit the number of participants and insist upon advance preparation.* Meetings work best when there are few participants – there are very good reasons why Fagan stipulated the precise membership of the inspection. Moreover, preparation for the inspection is crucial, so that the focus of the inspection – to find defects – is not lost.
- 7 *Develop a checklist for each product to be reviewed.* Checklists, by their nature, are blunt instruments. However, in that they facilitate focusing on major defects, checklists are also useful tools.
- 8 *Allocate resources and schedule time for inspections.* Inspections can be very cost-effective ways of improving product quality. However, they are not no-cost options and require a large investment of time. The pay-off is that they find defects early in the development process, when they are cheaper to correct. Proper training, adequate preparation time, and a meetings schedule are all necessary components of the initial investment.
- 9 *Provide meaningful training for participants,* as part of that investment.
- 10 *Review early inspections.* It is unlikely that a smooth path to inspection heaven will be gained immediately. Make sure that the most is gained from inspections, by learning from their successes and failures.

One final piece of advice on inspections: most blue-chip companies use inspections of some form!

These rules are adapted from *Software Engineering: A Practitioner's Approach* by Roger Pressman (2001).

SAQ 18

What are the benefits of conducting inspections for:

- (a) the product under inspection?
- (b) the development process for that product?
- (c) the inspection process?

ANSWER.....

- (a) The quality of the product under inspection will benefit by having defects in the product found and removed. The product itself may well also be ready for delivery earlier as far fewer defects are found downstream, for example during testing.
- (b) Keeping statistics on the types of defects found by inspections could pin-point problems in the development process, by indicating common defects in all products of a particular phase, for instance.

- (c) The development of checklists for each of the uses of inspection is one way in which the inspection process could benefit; the on-the-job training of inspection personnel is another.

You may well have listed other equally valid benefits in each case.



Exercise 9

What would the entry and exit criteria be, when conducting a review of a class diagram?

Solution

A suitable entry criterion for a class diagram might be that it is syntactically correct, as determined by a modelling tool. However, other criteria will also be needed depending on the stage of development process: at an early stage, we may simply require that enough classes are present to cover all the use cases we are going to implement; at the implementation stage, we may require that all operations are present. Our exit criteria is always the same, that is, the removal of defects, although the precise meaning of this may change.

6.3 Summary of section

Formal technical reviews, or inspections, are an increasingly popular technique for improving product quality. Unlike testing, inspections are very widely applicable to all products of the software development process, including high-level designs and test plans, and can improve each of them. This makes them an important technique for the professional software engineer to be aware of.

It is worth noting that there are less formal techniques that provide some of the benefits of inspections. For example, *pair programming*, a technique where two people work side by side on the same code, can be thought of as a form of continuous inspection.

7

Summary

In this unit you have studied quality and its relationship to the products of the software development process. You saw the importance of concentrating on producing products of appropriate quality, that is, products that are fit for their purpose and of sufficiently high quality so that they meet the customer's requirements and expectations. A number of software quality factors and metrics were introduced as ways of measuring whether a software product is of appropriate quality.

The unit also introduced verification and validation as ways of assessing the quality of a software product, and you saw that testing is the most important way of achieving verification and validation. The testing process, comprising unit, integration, system, acceptance and regression testing, as applied to object-oriented systems, was described. You studied the two main techniques for software testing, black box and white box testing, and saw that, for object-oriented systems, a sensible testing strategy combines both techniques.

The unit concluded with a discussion of the ways that formal technical reviews, or inspections, are useful in the development of quality software products.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ define quality and describe how the need for quality arises;
- ▶ describe how quality relates to the products of the software development process;
- ▶ describe how the customer's requirements and those from other sources can make demands on the qualities of a software system;
- ▶ define verification and validation, and explain how they can be achieved;
- ▶ describe testing, its relationship to verification and validation, and what it can (and cannot) achieve;
- ▶ describe the basic types and stages of testing;
- ▶ explain how software metrics allow the estimation of values for software quality factors;
- ▶ explain how software complexity can be measured in object-oriented systems;
- ▶ understand the role of black box and white box testing in the development of object-oriented systems;
- ▶ appreciate the limitations of black box and white box testing;
- ▶ develop a simple test strategy for both black box and white box testing;
- ▶ understand the purpose and benefits of inspections, and the inspection process.

References

- Beizer, B. (1996) *Software System Testing and Quality Assurance*, International Thompson Computer Press.
- Beizer, B. (1990) *Software Testing Techniques*, 2nd edn, Van Nostrand Reinhold.
- Boehm, B. (1984) 'Verifying and Validating Software Requirements and Design Specifications', *IEEE Software*, vol. 1, no. 1.
- Budd, T. (2000) *Understanding Object-Oriented Programming with Java: Updated Edition*, Addison Wesley.
- Chidamber, S.R. and Kemerer, C.F. (1994), 'A metrics suite for object-oriented design', *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–93.
- Curtis, B., Sheppard, S.B., and Milliman, P. (1979) 'Third time charm: stronger predications of programmer performance by software complexity metrics', *IEEE Proceedings of the Fourth International Conference on Software Engineering, 1979*, IEEE.
- D'Souza, D. and Wills, A.C. (1998) *Catalysis: Objects, Frameworks, and Components in UML*, Addison-Wesley.
- Fagan, M.E. (1976) 'Design and code inspections to reduce errors in program development', *IBM Systems Journal*, no. 3, pp.182–211.
- Feuer, A.R. and Fowlkes, E.G. (1979) 'Relating computer program maintainability to software measures', *Proceedings of the 1979 National Computer Conference*, AFIPS Press.
- Gilb, T. (1988) *Principles of Software Engineering Management*, Addison-Wesley.
- Littlewood, B. (1992) 'Software reliability modelling', *Software Engineers Reference Book*, edited by J.A. McDermid, Butterworth Heinemann.
- McCabe, T.J. (1976), 'A complexity measure', *IEEE Transactions of Software Engineering*, vol. 2, pp. 308–20.
- McCall, J.A. and Cavano, J.P. (1978) 'A framework for the measurement of software quality', in *Proceedings of ACM Software Quality Assurance Workshop, November 1978*, ACM.
- Office of Safety and Mission Assurance (OSMA) (1993) *Software Formal Inspections Guidebook*, NASA.
- Pressman, R. (2001) *Software Engineering: A Practitioner's Approach*, European Edition, McGraw-Hill.
- Russell, G.W. (1991) 'Experience with inspection in ultralarge-scale developments', *IEEE Software*, vol. 8, no. 1, pp. 25–31.

Index

A

acceptance testing 18, 24

accuracy 27

analytical completeness 14

appropriate quality 6

auditability 27

author 49

B

basis-path testing 40

black box testing 36

bottom-up testing 19

boundary testing 37

boundary-condition testing 19

C

case analysis 37

change impact analysis 24

cohesiveness 32

communication commonality 27

completeness 13, 27

complexity 27

concatenated loops 44

conciseness 27

consistency 27

control-structure testing 19

correctness 7

coupling-between-objects (CBO) metric 32

cyclomatic-complexity metric 30

D

data commonality 27

decision points 31

depth-of-inheritance-tree (DIT) metric 32

developmental testing 17

E

efficiency 7

entry criteria 49

error guessing 37

error tolerance 27

error-handler testing 19

execution efficiency 27

exit criteria 49

expandability 27

F

fence testing 37

flexibility 7

follow-up 50

G

generality 27

getter 19

H

hardware independence 27

I

initialisation testing 23

input data space 37

input domain 37

instrumentation 27

integration testing 18, 20

integrity 7

interface testing 19, 23

interoperability 8

L

lack-of-cohesion-in-methods (LCOM) metric 32

leaf 19

lines-of-code (LOC) metric 30

local-data testing 19

loop testing 43

M

maintainability 7

major defect 50

mean time to change (MTTC) 10

metrics 27

minor defect 50

moderator 49

modularity 27

N

nested loops 43

number-of-children (NOC) metric 32

O

object-oriented metrics 32

operability 27

operator testing 23

overview 50

P

partitioning 37

performance testing 23

planning 49

portability 8

preparation 50

product operation requirements 7

product revision requirements 7

product transition requirements 7–8

protocol testing 23

Q

quality 6

R

random testing 37

reader 49

regression testing 18, 24

reliability 7

report 50

requirements-based testing 17

response set 32

response-for-a-class (RFC) metric 32

restart testing 23

reusability 8

review 50

reviewers 49

rework 50

robustness 27

S

scribe 49

security 27

security testing 23

self-documentation 27

setter 19

simple loops 43

software metric 27

software quality factor (SQF) 7

start-up testing 23

straight-through path 31

stress testing 23

subdomain 37

system description 12

system independence 27

system test matrix 23

system testing 18, 23

T

test coverage 44

test harness 19

test stubs 19

testability 8

testing 15

traceability 27

training 27

U

unit testing 18

usability 7

usability testing 17

user-command testing 23

V

validation 12

verification 12

W

weighted-methods-per-class (WMPC) metric 32

white box testing 36